Talha

Date	Feature	Whats Done	Git Branch	Total Time Spent	Comments	Review
8 Dec 2025	**Access Portal Feature**	Setting up codebase + Understanding frontend architecture	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github account	**8 hours**		@Khurram Yousaf
9 Dec 2025	**Access Portal Feature**	Setting up codebase + Build issues fixed + Wrote guidelines for backend setup + Test DB setup	**https://github.com/Intimec/intimec-development-frontend**	**8 hours**		@Khurram Yousaf

			nd/tree/frontend-access-portal			
10 Dec 2025	**Access Portal Feature**	Enhance access portal UI and structure • Added sections for Request Access, My Requests, and Analytics • Updated tile layout and overall component formatting • Refined all tabs to align with the Lovable UI design system • Implemented nested tab structure for improved navigation	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github account	**8 hours**		@Khurram Yousaf
11 Dec 2025	**Access Portal Feature**	1. Standardized the sizing of dialogs across all tabs, including request dialog, edit tile, and add tile. 2. Optimized the height of the access portal for consistent	https://github.com/Intimec/intimec-development-frontend/tr	**8 hours**	Page selection by client (5,10,20,30) -	@Khurram Yousaf

		rendering across different devices. 3. Corrected and unified theme colors for both dark mode and light mode. 4. Implemented pagination for the Request Access tab. 5. Implemented pagination for the Requests tab. 6. Added a role-based selector for roles and departments, including an integrated search bar. 7. Added pagination support to the role-based selector. 8. Corrected and refined the tile card layout.	ee/frontend-access-portal Connect your Github account			
12 Dec 2025	**Access Portal Feature**	**Frontend** • Resolved dialog-related issues • Updated the internal role management dashboard • Project structure understanding • Fixed button click crashes **Backend** • Analyzed Identity Management (IDM)	https://github.com/Intimec/intimec-development-backend/tree/access-portal-backend	**8 hours**		@Khurram Yousaf

		requirements • Created UML, class, and sequence diagrams • Defined workflow • Initial branch creation and push • Initial creation of database tables	d Con nect y our Gi thub a ccoun t			
15 Dec 2025	**Access Portal Feature**	**Backend** Created the backend crud for the tiles, create for tile, edit for tile, update for tile, delete for tile, search for tile and tiles Tables Creation for the feature	http s://git hub.co m/Inti mec/in timec- develo pment -back end/tr ee/acc ess-p ortal-b acken d Con nect y our Gi thub a ccoun t	**8 hours**		@Khurra m Yousaf
16 Dec 2025	**Access Portal Feature**	1. Implemented **CRUD APIs for Cart Portal** (add, update, delete, fetch cart items) with proper validations and error handling.	http s://git hub.co m/Inti mec/in timec- develo	**8 hours**		@Khurra m Yousaf

		2. **Tested APIs for Tiles module** using Postman, verified request/response structure and edge cases. 3. Integrated **API Gateway for request routing** to Access Portal Service (validated headers, auth flow, and service mapping). 4. Updated **UI to remove unused components and obsolete flows** to improve performance and maintainability. 5. Made required **textual/content changes** across UI as per latest requirements.	pment -back end/tr ee/acc ess-p ortal-b acken d Con nect y our Gi thub a ccoun t			
17 Dec 2025	**Access Portal Feature**	1. Refactored the tile fetching and creation logic to improve code maintainability and efficiency. 2. Implemented comprehensive logging across the tile and cart APIs to enhance traceability. 3. Integrated the APIs with the API	http s://git hub.co m/Inti mec/in timec- develo pment -back end/tr ee/acc ess-p	**8 hours**		@Khurra m Yousaf

		Gateway, ensuring proper propagation of `User-ID` and `Tenant-ID` headers. 4. Tested the end-to-end functionality of the Cart and Tile APIs to validate correctness. 5. Dockerized the Access Portal Service for consistent deployment across environments. 6. Developed mappers to standardize API responses and ensure consistent data formatting. 7. Centralized constants in a dedicated file, eliminating hardcoded values and improving maintainability. 8. Added validation handling for duplicate and missing resources, increasing data consistency and reliability to approximately 99%.	ortal-backend Connect your Github account			
18 Dec 2025	**Access Portal Feature**	**Frontend:** 1. UI development for tiles and cart.	https://github.com/In	**8 hours**		@Khurram Yousaf

		2. Initial setup of auditing library for frontend integration. 3. Implementation of internal role-based access control for UI elements. 4. Ensured proper rendering and visibility of tiles based on user roles. **Backend:** 5. Backend pagination for tiles and cart. 6. Initial setup of auditing library for backend. 7. Integration of auditing processes in backend workflows (in progress). 8. Enforcement of role-based access control at the API level. **Integration / Overall:** 9. End-to-end role-based access control validation. 10. Coordination of frontend and backend pagination. 11. Auditing process integration across frontend and backend	timec/intimec-development-backend/tree/access-portal-backend			

		(phase-wise). 12. Preparing for next phase implementation of complete auditing features.				
19 Dec 2025	**Access Portal Feature**	• Integrated the application with the permissions service to enable secure inter-service communication. • Implemented validation logic for the add-to-cart functionality to ensure data integrity. • Structured and organized proper validation files for improved maintainability. • Added centralized exception and error handling to gracefully manage runtime failures and improve API reliability. • Enforced role-based access control, ensuring users can access only the tiles permitted by their assigned roles. • Performed REST API testing using	https://github.com/Intimec/intimec-development-backend/tree/access-portal-backend Connect your Github account	**8 hours**		@Khurram Yousaf

		appropriate tools to validate request/response flows and edge cases. • Verified API error scenarios and response codes to ensure compliance with defined API contracts.				
22 Dec 2025	**Frontend and Backend Access-Portal**	**Progress Update** • Revamped the UI by replacing dialog-based components with a sheet-based interface that slides in from the right. • Migrated the Tile Management dialog, Request dialog, and Add-to-Cart dialog to the new sheet-based layout for improved user experience. **Backend Development** • Designed and implemented request handling logic, including DTOs, controllers, services, and their corresponding implementations.	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal	**8 hours**		@Khurram Yousaf

		• Developed the service, repository, and mapper layers following clean architecture principles. • Implemented and tested APIs to ensure correctness, reliability, and seamless integration with the frontend.				
23 Dec 2025	**Frontend and Backend Access-Portal**	1. Implemented **Kafka Consumer Service** to process incoming events reliably. 2. Developed **Kafka Worker Service** for asynchronous event handling and background processing. 3. Added **Audit Logs** to track tile **creation, update, and deletion** actions for traceability. 4. Implemented **event validation** before pushing messages to Kafka to avoid invalid payloads. 5. Fixed **UI layout issues** and improved overall screen	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github account	**8 hours**		@Khurram Yousaf

		alignment and responsiveness				
24 Dec 2025	**Frontend and Backend Access-Portal**	• Implemented validations for tile configuration and visual filters • Integrated tile fetch APIs from the backend • Identified and resolved data mapping issues caused by required transformations across multiple frontend and backend files • Fixed auditing-related bugs • Refactored and centralized constants for better maintainability • Added clear comments to backend code structure for improved readability • Documented the work completed so far	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github account	**8 hours**		@Khurram Yousaf

25 Dec 2025	**Frontend and Backend Access-Portal**	1. Completed frontend integration with tile management APIs, replacing mock responses 2. Resolved visual filter update bugs, including rule chaining and field validation 3. Performed integration testing to validate API behavior and edge cases 4. Added loading indicators during tile fetch and interaction 5. Implemented add-to-cart functionality for tile selection 6. Conducted testing using complete datasets across multiple scenarios	https://github.com/Intimec-development-frontend/tree/frontend-access-portal Connect your Github account	**8 hours**		@Khurram Yousaf
26 Dec 2025	**Frontend and Backend both access portal**	1. Implemented **unified logging** across the application for consistent and traceable logs. 2. Created **sample demo videos** demonstrating tile **creation, update, and deletion** workflows.	https://github.com/Intimec-development-frontend/tree/frontend-	**8 hours**		@Khurram Yousaf

3. Integrated **access items with role-based permissions** to control feature visibility and actions.
4. Completed and thoroughly **tested full CRUD functionality for tiles**, including validation of each field and view scenarios.
5. Migrated the **detail modal** to the **Access Detail Sheet** for improved usability and alignment with design standards.
6. Debugged and validated **API requests** using multiple sample input scenarios.
7. Fixed and standardized **Request DTOs and Response DTOs** to ensure data consistency across services.
8. Developed **APIs for access portal item permissions**, along with **Next.js API routes** and client-side data fetching.

acces s-port al Con nect y our Gi thub a ccoun t

		9. Implemented **toast notifications** for tile creation, update, and deletion events. 10. Added robust handling for **loaders, empty-state scenarios, and graceful error fallback mechanisms** to enhance user experience.				
29 Dec 2025	**Frontend and Backend both access portal**	1. **Completed end-to-end CRUD testing for Tile Creation**, including robust **duplicate scenario handling**, ensuring data integrity and preventing inconsistent records. 2. **Implemented backend validations** to enforce business rules early, reducing invalid requests and improving API reliability. 3. **Delivered Add-to-Cart functionality**, enabling seamless user flow for access item selection and improving overall	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github account	**8 hours**		@Khurram Yousaf

		request submission experience. 4. **Eliminated mock data for access items** and replaced it with real data handling, addressing multiple **edge cases** to align behavior with production-like scenarios. 5. **Refactored constants into** `AccessPortalConstants`, improving code maintainability, consistency, and reducing duplication across the codebase.				
30 Dec 2025	**Frontend and Backend**	1. Integrated **Access Request History** API with frontend, including pagination and status-based filtering. 2. Implemented **Add to Cart** and **Remove from Cart** functionality with full backend–frontend integration. 3. Enhanced **Tile Access Scope Items** to support `allowRequestForOthers`, enabling	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github a	**12 hours**		@Khurram Yousaf

		conditional request targeting (self vs others). 4. Added **Department** field to Tile entity and ensured it is propagated to relevant APIs and UI components. 5. Implemented **Target Selection Modal** to open only when the tile allows requesting access for others. 6. Completed integration for: ◦ Fetching cart items ◦ Adding items to cart ◦ Removing items from cart 7. Added comprehensive **JavaDoc documentation** across the codebase for better maintainability and clarity. 8. Optimized **Request Access API** performance by reducing lookup complexity from **O(n²)** to **O(n)** using a	ccoun t			

		precomputed map-based approach. 9. Handled critical **access request edge cases**: ◦ Prevented users from requesting roles already assigned to them. ◦ Ensured already-assigned roles are hidden from backend responses. ◦ Validated duplicate role requests at both **Add to Cart** and **Request Submission** stages. ◦ Implemented frontend handling for backend validation errors with proper user feedback.				
31 Dec 2025	**Frontend and Backend**	1. **Resolved critical MultiBagFetchException** impacting request detail retrieval by identifying and refactoring conflicting entity fetch strategies, significantly	http s://git hub.co m/Inti mec/in timec- develo pment -front end/tr	**8 hours**		

		improving request detail API stability and reliability. 2. **Successfully integrated a comprehensive Request Details feature**, including: ◦ End-to-end request information ◦ Full audit log visibility ◦ Approval lifecycle tracking ◦ Item-level request metadata This provides a complete and traceable view of access requests for both users and reviewers. 3. **Implemented robust validation for the Request Details view**, ensuring data integrity, preventing inconsistent states, and improving overall user experience during request inspection. 4. **Standardized justification behavior across the application** by removing the	ee/frontend-access-portal Connect your Github account			

`allowJustification` option for "Others" in the dialog, ensuring consistency between request creation, update, and view flows.

5. **Completed a full rebase with the latest codebase**, resolving complex merge conflicts across:
   - API Gateway services
   - Docker Compose configurations
   - Environment credentials mismatches
   This effort stabilized local and deployed environments and ensured alignment across all dependent services.
6. **Finalized and pushed the fully integrated backend changes to the Access Portal Service**, unblocking downstream frontend integration and enabling end-to-end

		request lifecycle functionality.				
1 Jan 2026	**Frontend and Backend**	1. **Fixed audit logging for tile-related actions** by introducing a clear and consistent target object mapping, improving traceability and accuracy of audit records across tile operations. 2. **Integrated Withdraw Request functionality end-to-end**, ensuring: ◦ Successful request withdrawal ◦ Proper cart item state updates post-withdrawal This restored data consistency between requests and cart state. 3. **Implemented server-side pagination for Tile items**, optimizing fetch performance and improving scalability when handling large tile datasets. 4. **Developed and integrated a**	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github account Backend branch also	**8 hours**		@Khurram Yousaf

Withdraw Request dialog, enabling users to provide a mandatory withdrawal reason, improving auditability and compliance alignment.

5. **Resolved analytics module crashes** by stabilizing mock data handling, ensuring uninterrupted analytics rendering during development and testing.
6. **Enhanced loading experience for Tile listings** by introducing:
   - Skeleton-based loaders
   - Improved shadow and blur effects during pagination transitions
     This significantly improved perceived performance and UI polish.
7. **Added comprehensive documentation across the platform**, including:

		◦ Confluence documentation for Cart, Tiles, and Access Request flows ◦ In-repo backend documentation using structured `.md` files This improves onboarding, maintainability, and cross-team knowledge sharing.				
2 Jan 2026	**Frontend and Backend**	1. Completed merging and rebasing of the entire codebase. 2. Synced with Aymen to align on current development state. 3. Explored the codebase to identify non-functional areas and issues. 4. Discussed technical approaches in depth during the cross-functional meeting. 5. Integrated OpenSearch for logging and monitoring. 6. Fixed tile visibility parameters to ensure correct UI rendering.	https://github.com/Intimec/intimec-development-frontend/tree/frontend-access-portal Connect your Github account	**8 hours**		@Khurram Yousaf

		7. Corrected exception response codes for consistent error handling. 8. Updated request mapper for access requests to handle payloads correctly.				
5 Jan 2026	**Frontend and Backend**	• Fixed environment variables for Docker files to ensure consistent setup across environments. • Investigated integration between User Service and Identity Service for employee login functionality. • Ensured thorough testing approach for User Service with proper management of test scenarios. • Identified a solution for integration and testing that works seamlessly across all services. • Reviewed potential conflicts in service interactions and planned mitigation strategies. • Documented findings and integration	https://github.com/Intimec/intimec-development-backend	**8 hours**		@Khurram Yousaf

		approach for team reference. • Coordinated with team member to validate the proposed solution before implementation. • Analyzed service dependencies to ensure smooth end-to-end functionality.				
6 Jan 2026	**Backend**	Designed the solution for **Auth Service ↔ Identity Service integration**. Evaluated **REST vs gRPC vs Kafka**, finalized **gRPC + Kafka** based on trade-offs (performance, decoupling, scalability). Aligned on the complete implementation plan with the team. Set up **Auth Service** and **Identity Service** locally with all environment configurations.	https://github.com/Intimec/intimec-development-backend	**8 hours**		@Khurram Yousaf

		Tested Identity Service APIs; identified an issue where **employee creation API was failing**.				
7 Jan 2026	**Backend only**	Implemented **Auth and Identity Service integration**. Designed **Kafka topics** for **Identity lifecycle events**. Implemented **Kafka producer** in Identity Service for lifecycle events. Implemented **Kafka consumer** in Auth Service to consume identity events. Flow implemented: **POST create employee → Identity event published → Auth/User service consumes event → User created and linked**.	https://github.com/Intimec/intimec-development-backend	**8 hours**		@Khurram Yousaf

		Updated **User table schema** to include `identityId` and `identityType`. Replaced REST/gRPC-based creation with **Kafka-based async communication** for: 1. Better **fault tolerance** 2. **Loose coupling** 3. **Reliable and scalable event-driven flow**				
8 Jan 2026	**Backend**	1. Successfully integrated the authentication service with the identity service, enabling functional role assignment. 2. Implemented login functionality allowing users to authenticate as employees or non-employees via API calls. 3. Set up the initial codebase structure for the website to support upcoming feature development. 4. Reviewed and gained understanding of key components of the	https://github.com/Intimec/intimec-development-backend Connect your Github acc ount	**9 hours**		

		connector architecture. 5. Identified and resolved a critical issue related to employee creation events. 6. Ensured system resilience by validating that when the authentication service goes down and comes back online, it correctly consumes pending events. 7. Verified that employee creation events are processed reliably after service recovery. 8. Conducted end-to-end testing to confirm seamless interaction between auth and identity services. 9. Improved overall stability and reliability of the authentication.				
9 Jan 2026	**AI nexus main website - Frontend**	**Impact:** Improved UI consistency, fixed broken visuals, reduced clutter across the site. **Work done (≈36 pages):**	https://github.com/Kyousaf-intime/AIde	**9 hours**		@Khurram Yousaf

		• 100+ textual fixes (removals, replacements, copy corrections) • Fixed padding, column structure, and layout issues • Corrected logos, icons, favicons • Fixed contact form issues • Added missing CTAs (e.g., *Click More*) **Code impact:** • **+421 lines added** • **−1,597 lines removed** (major cleanup & dead code removal)	ntX-Website Connect your Github account			
12 Jan 2026	**AI nexus main website - Frontend**	**Impact:** Unified theme, stronger branding, better visual engagement. **Work done (all pages):** • Applied theme-level fixes across the entire site • Additional textual refinements & replacements • Added background video integration • Logo fixes and alignment consistency	https://github.com/Kyousaf-intime/AIdentX-Website Connect your Github account	**9 hours**		@Khurram Yousaf

		Code impact: • **+1,000 lines added** • **−400 lines removed**				
13 Jan 2026	**AI nexus main website - Frontend**	Fixed further theme changes across remaining pages Fixed navigation across all pages Fixed the structure and removal of wording further across all sections Fixed the theme for components that was not right	http s://git hub.co m/Kyo usaf-i ntime c/AIde ntX-W ebsite Conne ct you r Gith ub ac count	**9 hours**		@Khurram Yousaf
14 Jan 2026	**AI nexus main website - Frontend**	Web theme changes, Scroll to the top button, Header and Footer consistency, Version 3 remaining issues fixed, Video opacing fixing, Padding, Margin bugs across all pages, integrated chat component, fixed the see in action components for dashboard view, redesigned the hero component for consistency across all pages		**8 hours**		@Khurram Yousaf

15 Jan 2026	**AI nexus main website - Frontend**	Fixed the theme changes for the footer, refactored the hero sections for video rendering, integrated the contact api first made in spring boot then refactored back to the express js for vercel deployment, tried to fix the video aspect ratio issues, fixed the deployment bug for vercel serverless function		**9 hours**		@Khurram Yousaf
16 Jan 2026	**AI nexus main website - Backend + Frontned**	Integrated the backend for contact us, calendly integrated, fixed ui bugs,		**8 hours**		
19 Jan 2026	**AI nexus main website - Frontend**	Fixed version 4 changes, integrated legal terms pages, fixed navigations in the footer, rebased the frontend and backend of intimec, rebased the backend and frontend codebase		**8 hours**		
20 Jan 2026	**Access Portal Service - Backend**	1. Merged the latest codebase into the **development branch**. 2. Integrated **Auth Service** with **Identity**		**10 hours**		@Khurram Yousaf

		Service using token-based authentication and updated the Auth Service accordingly. 3. Fixed and optimized **Docker Compose configuration** for the **Access Portal Service**. 4. Prepared an **implementation plan for user role assignment** using the **Workflow Service**. 5. Verified service dependencies and ensured proper environment configurations for seamless service communication. 6. Conducted preliminary testing of integrated services to ensure connectivity and functionality.				
21 Jan 2026	**Application Service-Backend**	• Analyzed multiple approaches for bulk role assignment handling and evaluated their pros & cons. • Finalized and implemented the **most scalable**		**8 hours**		@Khurram Yousaf

solution suitable for large-scale user–role mappings.

- Introduced a **Batch Job table** to track async processing, status, and progress.
- Linked batch jobs to **user-role assignments** for traceability and auditability.
- Implemented validation and processing logic in the **User Service** for bulk role assignments.
- Added a **Kafka-based Batch Worker** to process assignments asynchronously.
- Created required **DTOs for Kafka events** and batch status communication.
- Implemented chunk-based persistence to optimize DB writes and avoid memory pressure.

Result: Bulk role assignments now run asynchronously, are

		fault-tolerant, scalable, and observable via batch status tracking.				
22 Jan 2026	**Access Portal and Workflow Service**	1. Implemented workflow event publishing from **Access Portal**. 2. Integrated **workflow selection form** and completed the tile creation. 3. Updated backend **service + entity** to support a list of **beneficiaryIds**. 4. Performed backend + DB migration and added an **index on workflowId** for faster fetching. 5. Tested workflow role assignment event publishing → **notification is coming successfully**. 6. Currently fixing/debugging **Application Permissions API** for role assignment during testing phase. 7. Configured **Kafka properties** for workflow event publishing.		**8 hours**		@Khurram Yousaf

23 Jan 2026	**Access Portal Backend + Workflow Service + Application Permissions Service**	1. Refactored the **application permission API** to support **multiple users in a single request** instead of one-to-one mapping. 2. Designed and implemented a new **access-portal event** to handle **role assignment workflows**. 3. Added **frontend modal support** for selecting **multiple users at once** during role requests. 4. Enabled users to **request multiple roles** for themselves as well as **assign roles to other users as beneficiaries**. 5. Introduced a **Kafka event structure** with **role as the key** and **list of users as the value** for better scalability. 6. Added **validity period** and **justification fields per role** to support audit and approval requirements.		**8 hours**		@Khurram Yousaf

		7. Implemented **duplication checks** for multiple role assignments using **Set-based lookups (O(1))** for performance. 8. Used **Java Streams API** to build a **fast and clean data processing pipeline** for role-user mapping. 9. Integrated the **approval workflow**, where requests are sent to **role owners** before proceeding. 10. Handled **owner approvals** and ensured the workflow continues only after successful authorization. 11. Completed **end-to-end API testing** for **different user scenarios**, including multi-role and multi-user cases. 12. Verified successful **role provisioning** post-approval and ensured data consistency across services.				

26 Jan 2026	**Access Portal Backend + Workflow Service + Application Permissions Service**	1. Implemented **Kafka consumer** for Access Portal request provisioning flow. 2. Integrated Kafka events for **request auditing** across services. 3. Added event handling for **request details sync** to downstream systems. 4. Analyzed multiple workflow approaches for the **request lifecycle orchestration**. 5. Proposed an end-to-end solution from **Access Portal → Application Permissions Service**. 6. Documented solution **pros & cons** for each workflow approach. 7. Designed event-driven flow to ensure **seamless request state transitions**. 8. Implemented handling for **Approval** state using Kafka events.		**8 hours**		@Khurram Yousaf

		9. Implemented handling for **Rejection** state with proper event propagation. 10. Implemented handling for **Pending** state to support in-progress requests. 11. Ensured request lifecycle remains consistent across services via **Kafka-based coordination**. 12. Improved reliability by reducing direct service dependencies using **async event processing**. 13. Optimized role assignment flow and reduced processing time significantly. 14. Achieved **~40% performance improvement** in role assignment execution. 15. Validated critical cases end-to-end for correctness and stability.				
27 Jan 2026	**Auth Service for**	1. Designed and implemented **gRPC**		**8 hours**		@Khurram Yousaf

	User Identities	**proto for user identity lookup**. 2. Updated **pom.xml** configurations to support new gRPC/proto changes. 3. Implemented **auth-service gRPC logic** to fetch identities for given users. 4. Added **shared proto definitions to common-protos**, enabling reuse across services. 5. Integrated identity lookup into **access-portal service**. 6. Fixed **gRPC client configuration for Docker-based environments**. 7. Tested identity fetching flow with **specific users** to validate correctness.				
28 Jan 2026	**Auth service, Identity service, Docker, Access portal, workflows service**	1. Implemented **gRPC proto and service flow for manager lookup** using identity IDs. 2. Followed the **same standardized gRPC pattern** to maintain consistency.		**10 hours**		@Khurram Yousaf

		3. Updated **submit access request structure** to align with the latest format. 4. Enhanced event publishing logic to **push workflow events per role per user**. 5. Evaluated multiple approaches for event publishing and **selected the most scalable solution**. 6. Updated **frontend "Add to Cart" flow** to support **multiple user selections**. 7. Updated **workflow action templates** for role assignment. 8. Modified **workflow payload structure** to match updated backend processing. 9. Refactored **submit request and event logic** into a **modular, maintainable structure**.				
29 Jan 2026	**Acccess Portal Backend and Frontend**	1. Implemented comprehensive input validations across the application to ensure data accuracy and consistency.		**8 hours**		@Khurram Yousaf

		2. Fixed UI layout and alignment issues for **tiles** and **access request structures** to improve overall usability. 3. Added proper labels and field mappings for displayed data to enhance clarity and readability. 4. Implemented corresponding **backend validations** to enforce business rules and prevent invalid requests. 5. Standardized and restructured backend API responses for better frontend consumption. 6. Identified and resolved bugs in frontend files that were causing rendering and data-binding issues. 7. Enhanced the UI to clearly indicate users who already have access, avoiding duplicate or unnecessary requests. 8. Performed regression testing to ensure				

		existing functionalities were not impacted by recent changes.				
30 Jan 2026	**Access Portal Backend and Frontend**	1. Successfully implemented and tested the **cart item removal** functionality. 2. Fixed validation issues when users request access for themselves as well as for others (previously inconsistent for self-requests). 3. Corrected the **tile approval item count**, ensuring accurate reflection of pending and approved requests. 4. Improved the **View Details** screen for access request items, allowing users to see complete and accurate item information. 5. Enhanced frontend–backend integration to ensure validation and approval flows work seamlessly. 6. Prepared and documented the **complete Access**		**8 hours**		@Khurram Yousaf

		Portal Service documentation prior to request submission. 7. Conducted end-to-end testing of the access request workflow to validate stability and correctness. 8. Addressed minor UI and logic inconsistencies identified during final testing.				
2 Feb 2026	**Access Portal Backend and Frontend**	1. Configured backend support for the **request detail flow**, including: ◦ Approval metadata (who approved, when approved) ◦ End-to-end handling for **request approval → auditing → provisioning** ◦ Improved traceability for approval and provisioning steps (audit-ready) 2. Fixed multiple **Access Portal status bugs**:		**8 hours**		@Khurram Yousaf

- Ensured modal close properly resets state
- Cleared cached/selected users on modal dismissal to avoid stale data issues

3. **Rebased and synchronized** all major changes with the **workflow service** to keep backend and workflow logic aligned
4. Updated **Cart modal logic** to correctly handle **duplicate requests**:
   - Shifted uniqueness check from `roleItemId` → `cartId`
   - Allows same role to be requested again after withdrawal
   - Supports requesting the same role for different users (e.g., A/B/C → later D/E/F) without conflicts
5. **Technical changes**:
   - Updated DTOs to support new cart-based uniqueness

		◦ Modified validations accordingly ◦ Refactored submit-request flow to use cart logic ◦ Updated Access Portal related files to reflect these changes				
3 Feb 2026	**Access Portal Backend and frontend**	1. **Stabilized withdrawal workflow** by fixing critical bugs, improving validations, and handling edge-case exceptions to prevent runtime failures. 2. **Implemented event-driven architecture** using consumer–producer patterns for request details and request-status updates. 3. **Enabled role-based event consumption**, allowing users and managers to track request status accurately across different roles. 4. **Built approval tab logic** and verified behavior by testing with multiple users		**8 hours**		@Khurram Yousaf

		and a manager to ensure real-world role flows work correctly. 5. **Resolved serialization/deserialization issues** and introduced a **factory pattern** for event consumption to improve maintainability and extensibility. 6. **Standardized event payloads** by introducing a producer factory with consistent and relevant data types instead of raw strings. 7. **Improved system throughput** by configuring consumer **concurrency to 3**, ensuring better parallel processing without overwhelming the system.				
4 Feb 2026	**Access Portal Backend**	1. **Improved fault tolerance** by enabling acknowledgements for failed events, preventing unnecessary re-		**8 hours**		@Khurram Yousaf

consumption and duplicate processing.

2. **Implemented retry control mechanism** with a maximum retry window of **3 minutes** and **1-minute threshold**, avoiding infinite retries and system overload.
3. **Rebased changes with the latest workflow branch**, ensuring alignment with ongoing workflow updates and reducing merge conflicts.
4. **Added provisioning and auditing at each workflow step**, improving traceability, compliance, and debugging capabilities.
5. **Ensured correct event sequencing**, fixing the order of events so downstream consumers process actions in the intended flow.
6. **Identified and fixed approval inconsistencies**, especially around

		approval type and approver identification, improving data accuracy. 7. **Resolved a critical multi-bag exception** by refactoring queries using **subselects and Hibernate sets**, eliminating ORM runtime failures in production scenarios.				
5 Feb 2026	**Access Portal Backend**	1. Improved **approval workflow reliability** by validating multiple real-world user/role combinations. 2. Ensured accurate **approval routing** by testing single and multiple user-role mappings. 3. Eliminated approval mismatches by fixing **incorrect approval type resolution**. 4. Strengthened system resilience by adding **context and evidence bundle validation checks**. 5. Prevented downstream approval failures through		**8 hours**		@Khurram Yousaf

		defensive data retrieval logic. 6. Corrected **event consumption hierarchy**, ensuring events are processed in the intended order. 7. Reduced approval inconsistencies caused by **out-of-sequence event handling**. 8. Increased confidence in production readiness by covering **edge-case approval scenarios**.				
6 Feb 2026	**Access Portal Backend**	1. Stabilized the provisioning flow by making the **consumer fault-tolerant** for failure events. 2. Prevented system crashes by safely handling **provisioning failed scenarios**. 3. Fixed incorrect approval state transitions between **started and responded** states. 4. Ensured accurate approval lifecycle tracking, improving		**8 hours**		@Khurram Yousaf

		audit and traceability. 5. Increased user flexibility by making **justification optional** across the workflow. 6. Reduced friction in request submissions, improving **user experience**. 7. Implemented updated **request detail structure** aligned with upcoming UI changes. 8. Avoided rework by aligning **UI sketches and functional expectations** with Khurram early.				
9 Feb 2026	**Access Portal Frontend and Backend**	1. Implemented pagination for access request listing (backend pageable + frontend integration). 2. Added dynamic “items per page” selection component and connected it with API. 3. Updated backend response structure to support paginated metadata (total		**8 hours**		

		pages, total elements, etc.). 4. Rebased codebase and resolved major conflicts across 8 files. 5. Reconfigured request details workflow to align with new approval structure. 6. Redesigned complete Request Details UI (Tabs, Audit, Approval sections). 7. Implemented step-based approval display including role, approval status, timestamp, and response details. 8. Refactored backend DTO structure to support new approval step and audit data flow.				
10 Feb 2026	**Access Portal**	1. Rebased latest approval-related changes and ensured flow consistency. 2. Designed and implemented analytics backend (Controller, Service, Entity, DTOs). 3. Developed analytics metrics: total		**8 hours**		@Khurram Yousaf

		requests, approved, rejected, most requested item, and user-wise analysis. 4. Integrated analytics API structure to support dashboard consumption. 5. Fixed UI data formatting issues for consistent analytics display. 6. Corrected event sequence handling and mapped target user name per role dynamically. 7. Resolved Hibernate multi-bag fetch exception by converting lists to sets and applying subselect strategy. 8. Removed hardcoded role/user ID logic and refactored to use access request ID dynamically.				
11 Feb 2026	**Access Portal Frontend and Backend**	1. Integrated analytics APIs with frontend dashboard UI. 2. Configured API routes, hooks, components, and file structure for analytics module.		**8 hours**		@Khurram Yousaf

		3. Fixed audit logs DTO mapping to align with new approval step structure. 4. Updated approval step DTO to properly handle approver details and timestamps. 5. Handled edge-case configurations for breaking scenarios in approval workflow. 6. Added approver name mapping in approval history and audit logs display. 7. Ensured end-to-end data consistency between analytics, approval, and audit modules. 8. Validated and stabilized full approval flow from request creation to final decision with updated structure.				
12 Feb 2026	**Access Portal Backend and Frontend**	1. Implemented provisioning flow to automatically handle both approved and rejected users, reducing manual intervention.		**10 hours**		@Khurram Yousaf

		2. Updated request status logic to correctly reflect **Rejected** and **Provisioned** states for all users, ensuring accurate tracking. 3. Standardized status transitions to eliminate inconsistent request states across the system. 4. Added audit logs for provisioning **completion** and **failure** events to improve traceability and debugging. 5. Implemented handling for duplicate role provisioning across different users to prevent redundant processing. 6. Fixed incorrect status mapping for mixed scenarios where some users were approved and others rejected within the same request. 7. Ensured partial approval flows correctly update the overall request				

		status, improving workflow accuracy. 8. Strengthened backend validation to prevent invalid or unintended provisioning triggers. 9. Synced provisioning logic with frontend workflow to ensure consistent and real-time UI status updates. 10. Fixed badge color inconsistencies to match updated status states for better visual clarity in the UI. 11. Renamed status from **Failed** to **Cancelled** to better reflect business logic and improve user understanding. 12. Rebased with the latest backend and frontend changes to resolve conflicts and maintain workflow stability. 13. Improved overall reliability and consistency of the request lifecycle, reducing edge-case				

		failures and production risks.				
13 Feb 2026	**Access Portal Frontend and Backend**	1. Tested the complete request workflow with multiple role owners to ensure proper handling, accuracy, and stability across different approval scenarios. 2. Updated the repository layer to support multiple role owners, improving scalability and flexibility in role management. 3. Enhanced audit logs to provide clearer and more user-friendly messages, especially in cases where some requests are approved and others are rejected. 4. Improved modal responsiveness by updating sizes and layouts to work consistently across different screen resolutions and devices. 5. Designed and structured the		**8 hours**		@Khurram Yousaf

		controller layer for analytics to support future reporting and tracking requirements. 6. Implemented the repository logic for analytics, enabling efficient data handling and better performance. 7. Focused on improving system transparency and traceability in the Access Portal to support better monitoring and decision-making.				
16 Feb 2026	**Access Portal Frontend and Backend**	1. Successfully merged frontend and backend changes into the **dev** branch after resolving multiple conflicts, ensuring smooth integration and stability. 2. Improved the request cart UI by fixing sizing issues, making it more consistent and readable across different screen sizes.		**8 hours**		@Khurram Yousaf

		3. Added tooltips on the request cart and tiles page to improve usability and help users understand actions more clearly. 4. Enhanced the tile cards to be more user-friendly with better layout, spacing, and interaction. 5. Implemented input field limitations (up to 256 characters) to improve validation and prevent backend errors. 6. Integrated analytics into the dashboard, including request count, approved, rejected, and most requested items to provide better insights for decision-making. 7. Fixed a critical issue where items were being added multiple times due to double-click, improving system reliability and user experience. 8. Standardized padding and margins across modals to				

		ensure design consistency and a cleaner UI. 9. Improved overall responsiveness and UI consistency across key workflows in the Access Portal. 10. Focused on reducing user friction and enhancing performance in the request flow, supporting a smoother provisioning experience.				
17 Feb 2026	**Access Portal Backend and Frontend**	1. Implemented filters for request tracking to improve search and monitoring efficiency. 2. Standardized and enhanced the header design to maintain UI consistency across the request detail and tiles pages. 3. Refactored the tile and request card layouts to optimize spacing and reduce unnecessary white space. 4. Improved overall UI responsiveness and		**8 hours**		@Khurram Yousaf

		visual hierarchy for better user experience. 5. Added skeleton loaders across key sections to enhance perceived performance and loading feedback. 6. Integrated structured debugging logs across major modules to support easier troubleshooting. 7. Improved logging coverage to capture critical application flow and runtime events. 8. Introduced a unified debugging console tab for centralized log visibility. 9. Enabled log search and filtering across all access portal services, consumers, and configurations. 10. Categorized logs based on levels (info, warn, debug) for better traceability and faster issue resolution.				

		11. Enhanced monitoring capabilities to help track request lifecycle and identify system bottlenecks. 12. Focused on improving overall system observability and developer debugging workflow.				
18 Feb 2026	**Access Portal Frontend and Backend**	1. **Tile Card Standardization** – Align headers, spacing, and padding across all tile cards. Ensure consistent font sizes and text alignment. 2. **Request Card Fixes** – Adjust layout, spacing, and alignment of request cards to match design specs. Remove unnecessary borders or horizontal lines. 3. **Details Modals Implementation** – Add detailed modals for items/requests with proper content layout and scroll behavior. 4. **Permissions in Approval Tab** –		**8 hours**		@Khurram Yousaf

Integrate role-based permissions to control visibility and actions in the approval tab.

5. **Header Standardization – Tiles** – Ensure uniform header styling, font, and spacing across all tile views.
6. **Header Standardization – Request History** – Apply consistent headers and spacing to request history section.
7. **Header Standardization – Access Items** – Align access items headers with the portal's standardized header format.
8. **Remove Horizontal Lines** – Remove unnecessary dividers/horizontal lines across cards, modals, and sections for cleaner UI.
9. **Padding and Margins** – Apply consistent padding/margins

		across all cards, tiles, and modal content to improve readability. 10. **Modal Size Fixes** – Standardize default modal sizes for detail and request modals; set defaults (e.g., width/height) to improve UX.				
19 Feb 2026	**Access Portal Frontend and Backend**	1. **Tile Card Fixes** – Corrected layout, alignment, and spacing issues for all tile cards. 2. **Request Card Fixes** – Fixed visual inconsistencies and standardized formatting for request cards. 3. **Detail Role Modal Fixes** – Corrected content, layout, and behavior in role detail modals. 4. **Tooltips Across Cards** – Fixed inconsistencies and positioning of tooltips on all card types. 5. **Request Detail Modal Structure** – Reorganized the structure for better		**8 hours**		@Khurram Yousaf

		readability and consistency. 6. **Role Type Validity Added** – Implemented validation for role types within modals. 7. **Critical Bug Fix – Request Responsibilities** – Fixed a major bug affecting request responsibilities display and handling. 8. **Horizontal Lines Removed – Cards** – Removed unnecessary horizontal dividers from all card components. 9. **Horizontal Lines Removed – Request Details** – Cleaned up horizontal lines from request detail sections. 10. **Padding & Margin Standardization** – Applied consistent padding/margin across cards and modals for uniform spacing. 11. **Header Consistency** – Ensured headers in modals and cards				

		match portal standards. 12. **UX Improvements** – Minor layout and interaction fixes to improve user experience (e.g., modal resizing, spacing, readability).				
20 Feb 2026	**Access Portal Frontend**	**UI/UX Fixes** • Fixed audit log card style • Fixed horizontal lines in cart slideout • Fixed tile borders and colors across all tiles • Standardized loaders across all pages • Fixed empty state for permissions (added "no permissions" with icon) **Color Fixes** • Fixed risk color in cart slideout • Fixed risk color badge on access items page • Fixed risk badge color on request detail modal • Fixed risk type color on Access Detail Modal		**8 hours**		@Khurram Yousaf

		• Fixed status colors in approval tab • Fixed status colors in request cards **Guard Rail Implementation** • Added guard rail on cart slideout • Added guard rail on create/edit tile management dialog **Form Validations** • Added validation on all tile fields • Disabled buttons until required fields filled • Marked mandatory labels with red star (*) • Required fields: title, description, category **Loader Fixes** • Standardized loader for permissions • Standardized loader for details • Made loaders consistent across all pages				
24 Feb 2026	**Access Portal Backend and Frontend**	• Fixed the audit log to correctly capture IP address, user agent, description, and risk level during submit and withdraw requests.		**8 hours**		@Khurram Yousaf

		• Resolved the issue with self-request functionality. • Improved the audit log for tile updates by correcting the object description for better readability, saving the previous state using thread local, and implementing AOP with before and join point so updates work as expected. • Fixed tile object filtering so the correct data is saved. • Added a deletion guard that shows the impact before confirming tile deletion.				
25 Feb 2026	**Access Portal Backend and Frontend**	1. Fixed **Add to Cart** logic to prevent role assignment if the user already has the role during cart submission. 2. Added validation to avoid duplicate role assignment while submitting access requests. 3. Integrated Role API using REST client to verify whether users				

		already possess specific roles before processing requests. 4. Implemented backend-driven role validation flow to ensure accurate access control before cart and request submission. 5. Updated Access Portal frontend to reflect new role-checking logic and improved submission flow. 6. Standardized Risk theme colors across the application for consistent UI representation. 7. Created reusable risk color utility (`utils/riskColor.ts`) to eliminate manual styling and enforce design standards. 8. Updated `global.css` and refactored risk-related components to use centralized styling for maintainability and scalability.				

4 Mar 2026	**Dashboard Page Enhancements Done**	**Dashboard UI & Modal Updates – Key Points** 1. **Modal Enhancements** ◦ Fixed colors, paddings, and spacing for all modals. ◦ Standardized 10 modals to ensure uniform behavior and styling. ◦ Refined modal integrations for smoother interactions across the dashboard. 2. **Theme Support** ◦ Improved theme switching compatibility (light/dark modes). ◦ Ensured consistent application of theme styles across all dashboard components. 3. **UI Consistency & Layout** ◦ Applied consistent design patterns across dashboard components. ◦ Fixed alignment and spacing issues			@Khurram Yousaf	

		for 30 tabs and 30 visualizers. ◦ Improved overall visual coherence for better user experience. 4. **Impact / Benefits** ◦ Dashboard now maintains a unified and professional look. ◦ Easier future scaling and maintenance due to standardized components. ◦ Improved interaction quality and visual alignment for end-users.				
5 Mar 2026	**Access Catalog Page and Identities Page**	**1. Modal-to-Sheet Conversion (Critical UX Improvement)** • **30+ modals converted** to slide-in sheets across entitlement and role catalog views • **Improved context preservation** - Users maintain spatial awareness while accessing detailed information • **Consistent interaction pattern**		**8 hours**		

with right-side sheets for drill-downs, left-side for filters

- **Performance boost** through reduced DOM complexity and smoother animations
- **Mobile-responsive behavior** with full-width sheets on smaller screens

2. Visualization Standardization (30+ Components)

- **Unified sizing to 300px height** across all chart components:
  - Entitlement Intelligence graphs
  - Role Usage Coverage charts
  - Risk & Compliance visualizations
  - Ownership Classification diagrams
  - Sprawl & Usage analytics
- **Consistent color mapping** implemented across all visualizations:
  - Primary actions: `#0F172A` (slate-900)

    - Risk indicators: Semantic colors (red for high risk, amber for medium, green for low)
    - Status badges: Standardized palette matching design system
- **Improved readability** with standardized tooltip positioning and legend placement
- **Responsive scaling** maintaining aspect ratio across viewport sizes

3. Color System Refinement

- **Risk badge consistency** across 50+ entitlement and role components
    - High Risk: `destructive` variant with consistent red shades
    - Medium Risk: `warning` variant (new standardized amber)
    - Low Risk: `success` variant with green tones

- **Privilege level indicators** (Read/Write/Admin) with distinct, accessible color coding
- **Lifecycle status colors** standardized:
  - Active: Green (`bg-green-100 text-green-800`)
  - Deprecated: Slate (`bg-slate-100 text-slate-800`)
  - Orphaned: Orange (`bg-orange-100 text-orange-800`)
  - Inactive: Red (`bg-red-100 text-red-800`)

4. Access Catalog Cards Enhancement

- **12 card components** redesigned with consistent padding, shadows, and hover states
- **Interactive card grid** with smooth transitions and focus states
- **Information hierarchy** improved

		with standardized typography scale • **Action buttons** placement optimized for natural user flow • **Loading states** added with skeleton screens for better perceived performance **5. Progress & Impact Metrics** • **150+ files updated** across the codebase • **30+ visualization components** standardized • **50+ color variables** consolidated and aligned with design tokens • **100% of drill-down modals** converted to sheet components • **40+ bug fixes** related to color inconsistencies and sizing issues • **Performance improvement**: 15% reduction in initial bundle size through component consolidation				

6 Mar 2026	**Frontend**	**Component Standardization** • Introduced **StandardTabs component** under `shared` to be reused where **Directory and Analytics (2 tabs)** pattern exists. • Created reusable **PageHeader component** for consistent page headers across the platform. • Introduced **IconTabs component** for pages with **more than 2 tabs** to maintain UI consistency. **UI Responsiveness** • Improved **responsiveness across all screens** to ensure proper layout behavior on different breakpoints. • Standardized spacing for **sub-tabs** by applying `mt-4` margin for **IconTabs sections**.				

Identity & Access Pages

- Implemented **StandardTabs** for Identity pages where **Directory / Analytics tabs** are used.
- Completed **Non-Employee page tab migration** to standardized tab components.
- Added **Back navigation link** for the **Identities Group tab** for easier navigation.

UI & Visualization Fixes

- Fixed **Static vs Dynamic Distribution color mapping** in **Identities Group analytics**.
- Corrected **Department Distribution color consistency** in **Identities Group charts**.

Modal & UI Framework Updates

- Converted **modal dialogs to Sheets** for components generated from **Loveable UI**.

		• Fixed **risk badge color styling** for components originating from **Loveable-generated UI**.				
9 Mar 2026	**Frontend**	**UI & Layout Improvements** • Standardized page headers using the **PageHeader component** across multiple pages. • Added **breadcrumbs navigation** to employee and other key pages for better navigation. • Ensured **consistent header structure** (icon, title, description, actions). **Tabs & Navigation** • Removed icons from **Relationship** and **AI Analysis** tabs. • Converted several pages to **icon-based tabs** using the reusable `IconTabs` component. • Fixed duplicate **Access Events** tab and standardized **tabs to 4 columns** where required.		**8 hours**	@Khurram Yousaf	

- Updated **Application Permissions tabs** to icon-based tabs.

Components & Reusability

- Added **CardAction support in the Card component** to fix components referencing it.
- Created reusable **ListControlsCard** component for **search, filters, and pagination** in Responsibilities pages.

Access Portal

- Updated **Access Portal icon to** `KeyRound` for clearer access semantics.

Task Management

- Fixed **severity badge colors (High / Medium / Low)**.
- Improved **Task Management page headers**.
- Fixed **Quick Task Selection card tabs**.

Identity & Relationships

- Fixed the **Relationship tab** in

the Identity Details panel.

Organization Module

- Corrected **risk severity badge colors** in **KPI drilldown modals** (7 modals updated).
- Fixed **Create Snapshot modal spacing** in Organization Explorer.

Debugging & Audit Logs

- Added **standardized headers** to Debugging and Audit Logs pages.
- Introduced **icon-based tabs for log types**.

UI Consistency

- Standardized page padding across the platform:

```
p-4 sm:p-6 lg:p-8
```

- Fixed **padding alignment** in Debugging and Audit pages to match Dashboard, Identities, and Access Catalog.

		Dialog Fixes • Fixed dialogs for **Employee and Non-Employee flows**.				
10 Mar 2026	**Identiites Page, Access Catalog**	1. Updated **modals** across the application – standardized layout, adjusted spacing, and added modal descriptions for better clarity and usability. 2. Improved **UI consistency** by removing icons from all modals and applying a unified **4-spacing scale** inside modals and main Identity pages. 3. Applied **color theme updates** across **Identifiers, Identities, all tabs, and nested tabs** to align with the latest design guidelines. 4. Refined **tab structures** by updating styling and ensuring consistency across **primary and nested tabs**. 5. Fixed and aligned the **Relationship tabs** to match the latest		**8 hours**		

		implementation and design behavior.				
11 Mar 2026	**Auditing, Debugging, Monitoring, Sod Policies, Access Analysis, Dashboard Page**	**Analytics & UI Enhancements** • Added **Audit Logs Analytics** from Loveable implementation. • Implemented **Debugging Analytics dashboard**. • Added **Auditing Analytics**, including header and spacing improvements. • Imported **Access Review Analytics components** and fixed chart colors for three analytics charts. • Imported **SoD Policies Analytics and Directory components**. **UI & Design Fixes** • Fixed **chart color issues** to align with the theme. • Corrected **modal spacing inconsistencies**. • Updated **severity risk color styling** to match the application theme.				

- Fixed **debugging card layout and formatting**.
- Improved **auditing UI spacing and header structure**.

Debugging Module Improvements

- Fixed **Debugging Filters functionality**.
- Updated **Debugging UI card structure** for consistency.
- Integrated **Debugging Analytics section**.

Roles Catalog Improvements

- Updated **Roles Catalog tabs** to match the latest Loveable design.
- Fixed **role type bugs caused by missing null checks in Loveable code**.
- Fixed **tab styling issues in two locations** within the Roles Catalog.

Platform Integrations

- Imported **Monitoring page components**.
- Converted **React Router navigation to**

		Next.js navigation across the Monitoring module. • Identified **modal consistency fixes still required**. **Navigation Migration (React → Next.js)** • Replaced **React Router navigation with Next.js navigation** across **20 files**. • Updated routing hooks (`useNavigate` , `useLocation` , `useParams`) to Next.js equivalents.				
12 Mar 2026	**Access Analysis, Task Center, Access Portal**	**Access Analysis, Reports, Task Center, and Access Portal Updates** **Pages Imported** • Access Analysis page imported • Reports page imported • Task Center page imported from Loveable • Task Center filters fixed • Insights modal spacing corrected in Task Center				

UI Standardization

- Standardized card design across multiple modules
- Access Analysis cards updated to match standard UI
- Access Catalog cards standardized
- Reports page cards converted to KPI-style KPI cards
- Organization page cards updated to standard design

Layout & Spacing Fixes

- Fixed spacing issues in **Access Review page**
- Fixed spacing issues in **Main Access Catalog page**
- Fixed spacing issues in **Reports page**
- Fixed spacing issues in **Access Intelligence page**
- Fixed spacing issues in **Access Portal page**
- Fixed nested page spacing in **Access Review**

Access Intelligence Improvements

- Updated page structure
- Tabs moved before analytics cards to improve layout consistency

Component & Table Improvements

- Fixed **Entitlement Catalog page**
- Updated **Existing Role Models table design** in Access Analysis page
- Fixed **Entitlement Risk Distribution chart colors** that were rendering black
- Improved card styling and spacing in Access Analysis
- Fixed layout and UI issues in **Access Portal page**

Reports Module Updates

- Standardized Reports page card layout
- Fixed Reports page spacing
- Set **recommendations default value to false**

		Data Fixes • Fixed mock data for **Organizations tab** **Remaining Work** • Spacing adjustments across newly imported pages • Modal alignment and UI consistency fixes across new pages **Changes Summary** • UI consistency improvements across Access, Reports, Portal, and Task Center modules • Layout and styling fixes applied across multiple pages • **Total files modified: 78**				
13 Mar 2026	**Risk analytics, Lifecycle, Documentation**	**Summary of Changes** • **Codebase Update** ◦ ~23K lines changed across **171 files**. **UI Fixes** • Fixed **spacing issue in the "Create New Campaign" modal**. • Fixed **spacing on the Access Review main page**.				

		• Corrected **color issue in the Decision Breakdown chart**. **Feature Additions** • Added **Lifecycle tab** to the UI. • Implemented **Risk Analytics tab**. • Updated `en.json` to include new tab messages for: ◦ Lifecycle ◦ Risk Analytics. **Bug Fixes** • Resolved **rendering loop issue in Identity Form** caused by parent component re-render triggered by child component. **Documentation** • Added **documentation files generated from Loveable** into the project. • Fixed **documentation screenshots**. • Corrected **tooltip behavior in documentation**.				

14 Mar 2026	**Responsibilities, Dashboard**	1. Fixed card spacing and layout across Internal Roles, Business Roles, Tasks, and SoD Policies pages 2. Standardized button naming (e.g., “View Details”) across cards for consistency 3. Cleaned up role assignment text by removing incorrect wording (“Approve Role-Assignement”) 4. Refactored Internal Role Details modal (moved from hardcoded implementation to reusable component) 5. Improved modal UI (fonts, spacing, wording, and added descriptive content) 6. Replaced risk text with standardized **colored risk badges** across modals 7. Fixed severity badge color to align with global badge system 8. Integrated Role Permissions and Role Details tabs into task details modal		**8 hours**		

		9. Standardized tabs with icons and improved spacing/alignment 10. Updated item numbering UI from `#1` format to **circular themed badge** 11. Implemented analytics metrics for Responsibilities page (Total Responsibilities, Pending Tasks, Owned Roles) 12. Standardized permissions component using Access Portal pattern across Internal Roles / Responsibilities 13. Improved overall UI consistency (theme, spacing, typography) across role-related views				
`17 Mar 2026`	**Workflows, Dashboard, Settings Ai Analytics Page**	**Daily Progress Update** • Fixed modal sizing and height issues on Responsibilities page • Converted Dashboard modals to translations (5 modals + 3 dashboard files)		**8 hours**		

		• Integrated workflow tabs into main Workflows page; improved padding, spacing, and tab icons alignment • Enhanced Monitoring page UI (background, standard header, card borders, and “View Details” button design) • Implemented `ListControlCard` for dropdown-based controls • Refactored Templates page workflow design and standardized header bar • Fixed dialog styling for “View Template” (risk colors, header, spacing, and removed inconsistent dividers) • Converted “View Details” into proper modal; fixed spacing, tabs styling, and action button alignment • Improved workflow card hierarchy, spacing, and visual structure			

- Fixed Design tab spacing, margins, and layout consistency
- Standardized settings headers across all sub-pages
- Integrated AI Management modules; resolved rendering issues and merged into application AI tab

---

UI/UX Standardization & Enhancements

- Standardized **Filter Card design system** across modules (Access Items, Templates, Monitoring, Workflows)
- Converted all filter/search sections into **Card-based layouts with tooltips, responsive controls, and active filter chips**
- Added **truncate-safe Select components with tooltip support** to prevent overflow issues
- Unified **badge color system** using reusable utilities

(`getRiskBadgeClass` , severity mapping)
- Replaced hardcoded styles with **centralized design tokens and reusable helpers**

Charts & Theming Fixes

- Fixed chart color inconsistencies by correctly handling **CSS variables (HSL → hsl())**
- Standardized chart color usage across dashboards for consistency and dark mode support

---

Component & Architecture Improvements

- Introduced **IconTabs reusable component** for consistent tab design with icons
- Refactored imports using **barrel pattern (index.ts)** for better modular structure
- Improved **tooltip usage across UI** for better usability and clarity

- Enhanced **card UI components** with better hierarchy, spacing, and interaction states

---

Workflow & Permissions Enhancements

- Replaced custom permission UI with **standard RolePermissions component integration**
- Fixed scroll and layout issues in permissions & members tabs (full-height + controlled scrolling)
- Improved drilldown mapping and data handling for internal roles
- Fixed null/undefined handling for role data (e.g., `userCount`)

UX Improvements

- Fixed layout shift in workflow cards by ensuring **consistent "Last Activity" row (with "Never run" fallback)**

		• Improved visual consistency across cards (metrics, indicators, badges, spacing) • Added tooltip-driven micro-interactions for better user guidance • Enhanced responsiveness and alignment across filters, cards, and tabs				
18 Mar 2026	**Debugging Analytics Api and approach design**	1. Built four analytics APIs on audit logs returning KPIs, daily event trends, event type breakdown, and most active actors, all filtered by the logged-in user's organisation 2. Each API supports a date window or `range=all` to fetch full history with no date restriction 3. Top actors shows who did the most in the system, how many failed, and whether their overall activity was low, medium, or high risk 4. Risk level per actor is calculated from		**8.5 hours**		

actual event data, for example an actor with mostly low risk events gets a low badge even if they had a few high risk ones

5. Event volume returns one entry per day in the date window including zero-count days so charts never have gaps
6. Event distribution returns top 10 event types by default with a `limit` param to get 5, 25, or all
7. All APIs share a reusable window resolver, validator, and constants so adding a new endpoint follows the same pattern throughout
8. Projection interfaces map GROUP BY query results directly to response DTOs without loading full entities, keeping memory footprint low
9. Failure percentage uses BigDecimal rounding so the number shown is

		consistent across environments 10. Constants files act as a single source of truth for risk weights, thresholds, status literals, and log keys so nothing is hardcoded anywhere in logic				
19 Mar 2026	**Auditing, Debugging Analytics**	1. Built an **integrated debugging analytics system** combining frontend KPI tracking with backend log analysis. 2. Developed **interactive dashboards** with charts to visualize auditing data and system performance metrics. 3. Implemented **dynamic modals** to display detailed debugging insights and KPI drill-down information. 4. Designed frontend to track and display **real-time KPIs**, improving visibility into system behavior. 5. Created backend services using		**8 hours**		

		OpenSearch queries to fetch and analyze debugging logs efficiently. 6. Structured log queries to support **KPI-based filtering and aggregation** for faster issue identification. 7. Identified and fixed **field name mismatching issues** between logs and query responses, ensuring data consistency. 8. Validated debugging workflows by **testing multiple query scenarios** and confirming accurate results. 9. Followed **clean code principles** (modularity, readability, separation of concerns) across frontend and backend implementations.				
23 Mar 2026	**Analytics for debugging**	Updates • Implemented APIs for debugging KPI-related data		**8 hours**		

		• Fixed date handling based on timezone for consistent reporting • Added logging for error volume tracking • Created APIs to support heatmap data generation • Developed volume entry logs APIs for better data tracking **Impact** • Improved visibility into KPI performance and issues • Consistent and accurate time-based analytics • Better monitoring of system errors • Enhanced data analysis through heatmaps and logs				
24 Mar 2026	**Analytics for debugging**	• Enhanced existing analytics KPIs, added new metrics, and resolved multiple backend and query-related issues **Work Done** • Implemented 24-hour total error tracking		**8.5 hours**		

- Added 24-hour drawer logs view
- Removed date filters from KPI layer for consistency
- Refined KPI logic:
  - Error and warning aggregation
  - Average response time for info, warn, error logs
- Designed and implemented entries APIs:
  - Modal data APIs
  - Fixed modal APIs aligned with KPIs
  - Mean Time to Resolution (MTTR) API
  - Error trend API
  - Recurring error patterns API
  - Heatmap API
  - Volume count API
  - Category distribution API
- Backend responsibilities:
  - Designed OpenSearch queries for aggregations
  - Built scalable APIs for KPI calculations

- Handled edge cases in log data
- Frontend responsibilities:
  - Integrated APIs into routes and hooks
  - Connected analytics drawer with backend data
  - Updated dashboard for KPI rendering

Challenges / Debugging

- OpenSearch query issues:
  - Complex aggregation queries causing slow performance
  - Fixing nested aggregation structures
  - Ensuring correct grouping for logs
- Exception handling:
  - Missing/null fields in logs breaking aggregations
  - Standardizing response formats
  - Adding fallback handling in APIs
- 10k data limit issue:

		◦ Hit OpenSearch result window limit ◦ Avoided deep pagination ◦ Shifted approach to aggregation-based queries ◦ Evaluated scroll/search_after approaches **Impact** • Accurate and meaningful KPI data • Faster debugging using structured analytics • Stable and optimized OpenSearch queries • Improved dashboard performance				
25 Mar 2026	**Backend for workflow and F.E and B.E for responsibilities**	**Work Done** • Designed workflow-related KPIs: ◦ Total workflows ◦ Active workflows ◦ Draft workflows ◦ Success rate ◦ Failed workflows ◦ Average execution time • Built workflow analytics APIs: ◦ Execution trend data		**8 hours**		

- Success status breakdown (pending, failed, successful)
- Category distribution

Backend Responsibilities

- Designed and optimized workflow aggregation queries
- Implemented logic for success and failure rate calculations
- Computed average execution time efficiently across workflows
- Structured APIs for scalable and reusable analytics consumption
- Ensured data consistency across multiple workflow states

Challenges

- Aligning workflow data with existing analytics structure
- Handling multiple workflow states

		(pending, failed, success) in aggregations • Ensuring accurate and performant execution time calculations **Impact** • Enabled complete backend support for workflow analytics • Improved accuracy of workflow performance metrics • Provided scalable APIs for future analytics enhancements • Strengthened data foundation for workflow insights				
26 Mar 2026	**Workflow analytics**	**1. Backend Analytics API - Complete Service Layer Implementation** • **Impact:** Enables real-time workflow performance monitoring and data-driven decision-making for IAM lifecycle automation • **Technical:** Implemented 5 production-ready REST endpoints		**9 hours**		

		(`/analytics/overview` , `/execution-trend` , `/execution-distribution` , `/category-distribution` , `/kpi-drilldown`) with full service layer, repository queries, and UnifiedLogger integration • **Standards:** Zero hardcoded strings/magic numbers, all constants extracted to dedicated files, comprehensive JavaDoc on all methods **2. KPI Drilldown System with 8 Drill-Down Types** • **Impact:** Users can click any KPI card to view paginated, sortable workflow lists filtered by specific criteria (e.g., high-risk, failed executions, AI-driven) • **Technical:** Added new KPI types: `TOTAL_WORKFLOWS` , `ACTIVE_DRAFT_WOR`				

`KFLOWS`, `AUTOMATED_MANUAL_WORKFLOWS`, `SUCCESS_RATE_WORKFLOWS`, `AVERAGE_EXECUTION_TIME_WORKFLOWS`, `FAILED_EXECUTION_WORKFLOWS`, `HIGH_RISK_WORKFLOWS`, `AI_DRIVEN_WORKFLOWS`

- **Architecture:** Implemented switch-based query routing with dedicated repository methods (`findByStatusIn`, `findByTriggerTypeIn`, `findAllWithExecutions`, `findAllWithFailedExecutions`)

3. Time-Based Analytics Filtering with Configurable Periods

- **Impact:** Provides flexible time-window analysis (24h, 30/60/90 days, all-time) for trend identification and compliance reporting

- **Technical:** Implemented `calculateStartDate()` method with enum-based period constants, OffsetDateTime boundary calculations, and time-filtered repository queries (`countByStartedAtAfter`, `countByStatusAndStartedAtBetween`)
- **Example:** Success rate calculated over LAST_30_DAYS using `executionRepository.countByStatusAndStartedAtAfter(COMPLETED, startDate)`

4. Execution Trend Analysis with Monthly Aggregation

- **Impact:** Visualizes execution volume and failure patterns over 5-12 months to identify seasonal trends and capacity planning needs
- **Technical:** Built `YearMonth`-based iteration with configurable month

counts (5/6/7/8/10/12), month boundary calculations using `atDay(1).atStartOfDay()` and `atEndOfMonth().atTime(23,59,59)`, and chronological ordering (oldest→newest)

- **Data Format:** Returns `MonthlyExecutionDataDTO` with short month labels ("Jan", "Feb") and separate execution/failure counts per month

5. Category Distribution with Display Name Mapping

- **Impact:** Shows workflow distribution across lifecycle categories (JML, Access Request, Certification, etc.) with percentage calculations for portfolio balance analysis
- **Technical:** Implemented enum-to-display-name switch expressions (`JOINER_MOVER_LEAVER → "Joiner Mover Leaver"`),

percentage calculation with 2-decimal precision, descending sort by count

- **Business Logic:** Filters categories below minimum threshold, calculates percentages as `(count/total)*100` rounded to 2 decimals

6. Average Execution Time Calculation with Unit Conversion

- **Impact:** Identifies performance bottlenecks and SLA violations by surfacing workflows with longest completion times
- **Technical:** Added `getAverageExecutionTimeByWorkflowId()` repository method, milliseconds-to-hours conversion (`ms / (1000*60*60)`), 1-decimal precision rounding
- **KPI Enhancement:** `AVERAGE_EXECUTION_TIME_WORKFLOWS` drilldown

populates `avgExecutionTimeHours` field in DTO for sorting and display

7. Next.js API Route Layer with Authentication

- **Impact:** Secures analytics endpoints with session-based auth and provides clean separation between frontend and backend services
- **Technical:** Created 5 Next.js route handlers (`/api/workflows/analytics/*`) with `_getAuthToken()`, session ID propagation via `X-Session-Id` header, unified error handling pattern matching logging routes
- **Standards:** Consistent `try-catch` blocks, `.catch(() => ({}))` for error parsing, `cache: "no-store"` for real-time data

8. Custom React Hook with Granular Loading States

- **Impact:** Enables independent loading/error states per chart section, preventing UI blocking and improving perceived performance
- **Technical:** Built `useWorkflowAnalytics` hook with separate state variables (`overviewLoading`, `trendLoading`, `distributionLoading`, `categoryLoading`), auto-fetch on mount via `useEffect`, memoized callbacks to prevent unnecessary re-renders
- **UX Pattern:** Matches debug analytics dashboard pattern with `Loader2` spinners and per-component error boundaries

9. Interactive Analytics Dashboard with Drill-Down Modals

- **Impact:** Provides at-a-glance workflow health metrics with click-to-drill-down functionality for root cause analysis
- **Technical:** 8 KPI cards with click handlers, 4 charts (2 mock, 2 real), drill-down modal with pagination (page 0-indexed, pageSize max 100), CSV export, conditional column display for `avgExecutionTimeHours`
- **Data Sources:** Mixed real API data (KPIs, execution trend, distribution, category) and mock data (automation coverage, risk insights, efficiency table) with clear labeling

10. Repository Query Methods for Analytics

- **Impact:** Provides reusable, optimized database queries for all analytics

calculations with proper JPA repository patterns

- **Technical:** Added 15+ repository methods including `countByStatus`, `countByTriggerTypeIn`, `countByRiskScoreGreaterThanEqual`, `getAverageDurationAfter`, `findByStatusIn(List<String>, Pageable)`, `findAllWithExecutions`, `findAllWithFailedExecutions`
- **Performance:** Uses Spring Data JPA method naming conventions for automatic query generation, supports pagination and sorting, includes JPQL `@Query` annotations for complex aggregations

11. Comprehensive Constants Architecture

- **Impact:** Eliminates maintenance

nightmares from hardcoded values, enables consistent messaging across logs/errors, simplifies future i18n

- **Technical:** Created 5 dedicated constants files (`WorkflowAnalyticsConstants`, `WorkflowExecutionTrendConstants`, `WorkflowExecutionDistributionConstants`, `WorkflowCategoryDistributionConstants`, `WorkflowKpiDrilldownConstants`) with categorized sections (API endpoints, success messages, error messages, time periods, defaults, KPI types, logging field names, calculation constants)
- **Examples:** `HIGH_RISK_THRESHOLD = 7`, `PERCENTAGE_MULTIPLIER = 100.0`, `MS_PER_HOUR = 1000.0 * 60.0 *`

`60.0`, `LOG_FIELD_TIME_PERIOD = "timePeriod"`

12. Enhanced Tooltips with Data Source Transparency

- **Impact:** Users understand exactly what data they're viewing (real vs mock, time period, API source) for accurate interpretation and trust
- **Technical:** Updated all `InfoTooltip` text props to include: data source identification ("REAL DATA from execution-trend API" vs "MOCK DATA"), explicit time periods ("last 30 days", "last 6 months", "all-time"), status mapping details ("Success = COMPLETED", "Failed = FAILED + TIMEOUT + CANCELLED"), calculation formulas, and future roadmap hints
- **Example:** Success Rate tooltip now

		states: "Percentage of workflow executions that completed successfully without errors or timeouts in the last 30 days. Calculated from execution history filtered by LAST_30_DAYS time period. Target: >95% for production workflows. Data source: Real-time from execution-distribution API (last 30 days)."				
27 Mar 2026	**Login Page, Debugging Migrations, Responsibilities Analytics**	1. indexed OpenSearch collections in a controlled manner, followed by alias switching to ensure seamless adoption of new indexes. 2. Fixed inconsistent OpenSearch mappings in production by introducing index templates and reindexing ~5M+ documents, resolving field type conflicts and enabling accurate aggregations.				

		3. Optimized high-cardinality fields (`source`, `category`) by migrating from `text + keyword` to `keyword` type, improving query performance and consistency. 4. Implemented zero-downtime index migration strategy using reindexing and alias switching, maintaining backward compatibility with existing services. 5. Utilized Dokploy terminal to manage and persist indexed documents during migration processes. 6. Fixed UI issues in the login screen, including icon rendering, button alignment, branding text, and login button behavior. 7. Removed Critical Failures KPI from dashboard metrics.				
`30 Mar 2026`	**Debugging levels, Theme**	• Updated microservice log update service		**8 hours**		

		interface and implementation; separated constants into a dedicated structure • Implemented log level controller across all remaining 9 services • Added log level module under common/logging for identity service and configured package scanning in the Spring Boot application • Implemented log level controller for app permissions under common/logging and enabled scanning in the Spring Boot configuration • Resolved API gateway base URL conflict by updating the request mapping for the log level controller • Fixed authentication issues in identity and configuration services (approximately 45 minutes spent on				

		debugging) and validated log levels across all services • Added pagination support for correlation analysis and recurring errors module • Noted operational overhead: deployment (15–20 minutes), rebase/merge (5–45 minutes), and requirement to run mvn clean install for common protos and DAL core under the shared folder				
31 Mar 2026	**Api for entitlements mapping with resource names, theme context fixing, pagination for recurring errors on debugging page**	**Introduced Bulk Application Lookup API** • Designed and implemented `POST /api/v1/applications/bulk` • Enables fetching multiple application names using a list of `resourceIds` • Eliminates N+1 API call problem --- **2. Optimized Cross-Service Communication** • Replaced per-entitlement API calls				

with a single bulk request
- Reduced network overhead and improved response latency significantly

---

3. Created Lightweight DTO for Application Resolution

- Implemented minimal response DTO (`id` , `name`)
- Avoided heavy payload transfer from core service

---

4. Added REST Client in Entitlement Service

- Built `ApplicationClient` for calling core service
- Encapsulated HTTP logic and ensured separation of concerns

---

5. Enhanced EntitlementResponse DTO

- Added new field: `resourceName`
- Enables UI to directly display application name without additional calls

---

6. Implemented Bulk Enrichment Logic in Service Layer

- Extracted distinct `resourceIds` from entitlements
- Called bulk API once and mapped results
- Enriched response DTOs with application names

---

7. Maintained Clean Architecture Principles

- No business logic added to controller layer
- All enrichment handled inside service layer
- Reusable helper method for DTO transformation

---

8. Added Resilient Fallback Handling

- If application lookup fails, defaulted `resourceName` to `"Unknown"`
- Ensures entitlement APIs remain stable and do not fail due to dependency issues

---

		9. Logging & Observability Improvements • Added structured logs for: ◦ Bulk API calls ◦ Resolution counts ◦ Failure scenarios • Aligns with existing unified logging framework				
1 Apr 2026	**Frontend, backend**	**Backend** • Fixed the base URL configuration for hitting the correct app URL for app names. • Fixed unauthorized requests by attaching the authentication token to the Rest Client via a helper method that extracts the Bearer token from the current servlet request's `Authorization` header using `RequestContextHolder`. • Consolidated all hardcoded values into a dedicated constants file for maintainability. • Added Javadocs for the Rest Client class		**8 hours**		

and its methods.

- Tested the API for different pagination sizes and page numbers.
- Added the API route for fetching entitlements and fetching a single entitlement by ID.
- Added the API route for fetching members of an entitlement.

Frontend

- Created a custom hook for fetching entitlements, mapping the response, and defining TypeScript types for the entitlements response.
- Modified the Access Catalogue page to display entitlements sourced from the backend.
- Modified the entitlement view and editor updated the main layout and approximately 7–8 component files for each.

		• Added the entitlement members viewer tab that fetches and displays members of a selected entitlement. • Fixed risk badge casing to capitalize properly. • Fixed pagination sizing in the footer.				
2 Apr 2026	**Entitlements Analytics**	**Backend** • Built the analytics KPI endpoints for entitlements, including sprawl API and intelligence metrics. • Fixed the get-all-entitlements API to support filters: maximum users, minimum users, owner, classification, requestable, and assignment type (import, dynamic). • Cleaned up code, added comments, and wrote Javadocs across modified files. **Frontend** • Designed the sprawl KPIs section with mock data for sprawl		**8 hours**		

		index and stale entitlements, and real data for overprovisioned and direct assignments. • Designed the count-of-entitlements-per-application KPI card. • Fixed the entitlements table and search bar sizing to be consistent. • Removed the active badge from the display name column. • Added a tooltip for the description column in the entitlements table. • Fixed the risk badge rendering in the members tab. • Added labels for risk and status badges in the detail dialogue. • Added a detail panel view for members with corrected member name and email, and populated the required tabs (Overview, Accounts, Access). • Reduced font size for standardization across all badges.				

- Added member detail view on row click, matching the existing view button behavior.
- Removed icons from entitlement detail tabs and fixed the theme for entitlement history.
- Fixed the scripting viewer to be read-only and the scripting editor to allow editing.
- Tested dynamic role assignment and confirmed it works for the user.
- Removed the inner X button from the identity detail panel since the Dialog's built-in close button handles closing.
- Removed the unused `onClose` prop from the component destructure.
- Removed `bg-white border-l border-slate-200` from the outer div.
- Removed `border-b border-slate-200` from the header.

		• Removed `bg-slate-50/50` from the content area. • Removed `border-t` across the whole panel to eliminate unnecessary horizontal lines. • Removed unusual padding for consistency across all sections. • Fixed the inner tabs risk badge and status badge styling. • Removed non-standard icons from inner tabs. • Fixed risk badge and status badge text and color values.				
3 Apr 2026	**Entitlements Analytics**	**Backend** • Designed the risk KPIs with 1 real KPI (high risk) and 3 mock KPIs. • Designed the classification KPIs with 2 real KPIs (no owner and requestable) and 2 mock KPIs (requestability rate and unclassified). • Designed the risk distribution API that		**8 hours**		

returns the number of entitlements per risk level, currently returning zero because no risk-tagged entitlements exist yet, but the logic is real.
- Integrated the intelligence KPIs and entitlements-per-application endpoint.

Frontend

- **Intelligence tab:** All KPI cards are real. Application chart and its click entries are real. KPI card click entries are real. Type chart is mock but its click entries are real. Modal pagination is real.
- **Ownership / Classification tab:** Ownership Gaps and Requestable KPI cards are real. Requestability Rate and Unclassified KPI cards are mock. Ownership gaps, unclassified, and requestable click entries are real. Unowned preview list

		is real. Classification chart is mock but its click entries are real. Modal pagination is real. • **Risk / Compliance tab:** High Risk KPI card is real. SoD Exposure, Compliance Tagged, and Data Sensitive KPI cards are mock. High risk click entries are real. Risk distribution chart is mock (currently zero data) but its click entries are real. Compliance coverage list is mock but its click entries are real. Modal pagination is real. • **Sprawl tab:** All KPI cards are real. Stale, over-provisioned, and direct assignment click entries are real. Stale & orphaned preview list is real. Sprawl trend chart is mock. Modal pagination is real. • **Review tab:** All KPI cards, charts, lists, drilldowns, and				

		pagination data are mock.				
6 Apr 2026	**Rule evaluation logic, and testing**	**Backend** 1. Fixed "Executing an update/delete query" error by extracting rule evaluation into a separate Spring bean so transactions are not bypassed by self-invocation. 2. Fixed "Row was updated or deleted by another transaction" error by moving auto-evaluation to a TransactionalEventListener that fires only after the save transaction commits. 3. Replaced global match mode (ALL/ANY) with per-condition logical operators (AND/OR) enabling left-to-right expressions like (dept = Eng AND title = Senior) OR (dept = Product). 4. Enforced one rule per entitlement with a unique constraint to prevent duplicate rules that waste evaluation cycles.		**8 hours**		

5. Added case-insensitive field resolution so clients sending jobtitle, jobTitle, or JOBTITLE all resolve correctly and invalid fields are rejected at creation time.
6. Defaulted identity type to EMPLOYEE to prevent "field not allowed" errors when rules use employee-specific fields like department or jobTitle.
7. IMMEDIATE rules now auto-evaluate after creation and update without requiring a separate manual API call.

Frontend

8. Replaced all mock/hardcoded conditions in the entitlement detail view with live data from the assignment rule API.
9. Built rule creation and update flow in the visual builder using real backend

		field names, operators, and per-condition AND/OR logical operators. 10. Added enable/disable toggle switch that activates or deactivates a rule without deleting it. 11. Added proper loading, error, empty, and unsaved-changes states across the viewer and editor. 12. Created Next.js API route (GET/POST/PUT/PATCH) and React hook with createRule, updateRule, and toggleRule matching existing codebase patterns				
7 Apr 2026	**Rule Evaluation Entitlements**	Backend Changes --- **1. Designed and implemented scheduler logic for dynamic entitlement assignment rule evaluation** Built the scheduling infrastructure that allows entitlement assignment rules to be		**10 hours**		

		evaluated on different schedule types including IMMEDIATE, CRON, MANUAL, and ONE_TIME. The scheduler picks up rules based on their configured schedule type and triggers evaluation against all active identities of the matching identity type. Per-entitlement rules default to MANUAL schedule type where evaluation is triggered explicitly via the Run Now button from the UI. Impact: Entitlement assignments can now be automated on a schedule or triggered on demand, replacing the need for manual bulk assignment workflows. This is the foundation for continuous compliance where rules are re-evaluated periodically and assignments are granted or revoked automatically. Technology: Spring Boot, @Scheduled annotation, Quartz-style cron expressions, JPA				

Criteria API, PostgreSQL.

---

2. Implemented global trigger endpoint for changing schedule type across all rules

Created an admin-level global schedule endpoint that allows changing the schedule type for all assignment rules at once. This enables administrators to switch between MANUAL and CRON modes globally without editing each rule individually. The endpoint validates the schedule type and cron expression before applying the change across all active rules for the tenant.

Impact: Operations teams can quickly switch all rules to manual mode during maintenance windows or enable scheduled evaluation across the board without touching individual rule configurations. Technology: Spring

Boot REST controller, transactional batch update, JPA bulk update queries.

---

3. Changed schedule type from IMMEDIATE to MANUAL as the default for per-entitlement rules

Previously rules were created with scheduleType=IMMEDIATE which would trigger evaluation right after creation. Changed the default to MANUAL so that newly created rules do not auto-evaluate. Users must explicitly click Run Now to trigger evaluation. This prevents accidental mass assignments when a rule is still being configured or tested.

Impact: Safer rule creation workflow. Rules can be created, reviewed, and tested before any identities are affected. Eliminates the risk of incomplete rules running against production identities immediately on save.

Technology: Spring Boot service layer, DTO default values.

---

4. Implemented grouped conditions model with groupOperator and negated support

Replaced the flat condition list with a grouped conditions model where conditions are organized into groups. Each group has a groupOperator (AND/OR) that combines conditions within the group and a negated flag that wraps the entire group result in NOT. Groups are chained together using the previous group's groupOperator. This was the initial implementation before the interGroupOperator separation.

Impact: Enabled complex rule logic like "must be in Engineering AND Senior title" combined with "must NOT be a contractor with TEMP prefix"

which was impossible with flat conditions. Rules can now express inclusion, exclusion, and mixed logic in a single rule.
Technology: Java DTOs, JSONB storage in PostgreSQL, JPA Criteria API predicate building, in-memory condition evaluation.

---

5. Tested grouped conditions and discovered inter-group chaining bug

After implementing grouped conditions we ran comprehensive tests with multi-group rules. Testing revealed that when Group 1 had groupOperator=OR for its internal conditions, that OR was leaking into the inter-group chain causing Group0 OR Group2 instead of Group0 AND Group2. Identities that should not have matched were showing up in evaluation results. The evidence builder was also hardcoding

overallMatch=true regardless of actual evaluation results.

Impact: Identified two critical bugs through testing before the feature reached production. The SQL predicate was producing incorrect WHERE clauses and the audit evidence was unreliable for compliance reporting. Technology: PostgreSQL query analysis, manual test data with 30+ employees across multiple departments and email domains.

---

6. Added interGroupOperator field to separate intra-group and inter-group logic

Introduced a new interGroupOperator field on each condition group that controls how this group connects to the next group. The groupOperator now exclusively controls how conditions combine

within a group. This two-operator design resolves the bug where an OR group would leak its OR into inter-group chaining. The interGroupOperator defaults to AND if not provided, maintaining backward compatibility with existing rules.

Impact: Rules with mixed AND/OR groups now evaluate correctly. A group can use OR internally to match any of several values while still being AND-connected to the next group. This was the root cause of false matches in multi-group rules. Technology: Java DTO update, JSONB schema extension, backward-compatible normalization in service layer.

---

7. Fixed SQL predicate builder to use interGroupOperator for group chaining

Refactored buildGroupedPredicate in

		RuleEvaluationTransactionHelper to read interGroupOperator from each group when chaining groups together instead of using the previous group's groupOperator. Updated the GroupedCondition record to carry both operators. The predicate builder now constructs the correct WHERE clause where each group's interGroupOperator determines the boolean connector to the next group. Impact: Critical fix. The JPA Criteria query now produces the correct SQL for all group combinations. Without this fix, any rule with an OR group would match far more identities than intended, leading to incorrect entitlement assignments. Technology: JPA Criteria API, CriteriaBuilder.and/or predicate chaining, PostgreSQL query generation.				

8. Fixed evidence builder overallMatch always returning true

The buildEvidence method was hardcoding builder.overallMatch(true) for every identity regardless of whether the conditions actually matched. Refactored buildGroupedEvidence and buildLegacyFlatEvidence to compute the actual boolean result by evaluating all conditions in memory, applying group negation, and chaining group results using interGroupOperator. The computed result is now passed to the evidence builder.

Impact: Audit trail and evaluation results now accurately reflect match/no-match decisions. Previously every identity in the evaluation log showed overallMatch=true even when multiple groups failed, making it impossible to debug rule behavior or satisfy

compliance audit requirements. Technology: In-memory boolean evaluation, MatchEvidenceBuilder, GroupEvidenceResult record for per-group tracking.

---

9. Updated condition group normalization and legacy condition wrapping

Modified toConditionGroupMaps to normalize and persist interGroupOperator alongside groupOperator and negated when storing rule conditions as JSONB. Updated wrapLegacyConditions to include interGroupOperator=AND when auto-wrapping flat legacy conditions into a single group. This ensures that rules created with the old flat format continue to work without any data migration.

Impact: Zero-downtime deployment. Existing rules stored in the old

format are automatically handled. New rules use the full grouped format with interGroupOperator. No database migration required.
Technology: Spring Boot service layer, LinkedHashMap JSONB serialization, backward-compatible normalization.

---

10. Fixed listRules service method signature mismatch with controller

The controller was passing int page, int size, String sort but the service interface had been changed to accept Pageable. Reverted the interface and implementation back to match the controller signature. The Pageable is now constructed internally using PageRequest.of with `sort.by` defaulting to createdAt DESC when no sort parameter is provided.

Impact: Build blocker fix. The application would not compile without this change. The controller and service layer are now aligned on the same method signature. Technology: Spring Data PageRequest, Sort, Pageable.

---

11. Added comprehensive mock data for grouped condition testing

Created SQL insert scripts for 26 test employees across 7 test categories covering full match (5 employees), group 0 failure (4 employees with wrong department or title), group 1 failure (4 employees with wrong email domain), group 2 failure (3 TEMP contractors), multiple group failures (4 employees), and edge cases (6 employees covering permanent contractors, TEMP-named employees, inactive status, wrong

tenant, and blank fields). Used correct JOINED inheritance format with inserts into both identities and employees tables. Fixed NOT NULL constraint violation on email column by using non-matching domain values instead of NULL.

Impact: Enables systematic verification of all group combinations with a known expected result of 7 matching identities. Covers positive matches, single-group failures, multi-group failures, negation edge cases, inactive status filtering, tenant isolation, and blank field handling.
Technology: PostgreSQL, JPA JOINED inheritance table structure (identities + employees tables), epoch millisecond timestamps.

Frontend Changes

---

		12. Updated TypeScript types to support grouped conditions with interGroupOperator Added RuleConditionGroup and RuleConditionGroupRequest interfaces with groupOperator, interGroupOperator, and negated fields. Updated AssignmentRuleResponse to include conditionGroups array alongside legacy conditions array. Updated RuleCreateRequest and RuleUpdateRequest to accept conditionGroups as the preferred format. Removed logicalOperator from RuleCondition as it is replaced by group-level operators. Impact: Frontend type system now matches the updated backend API contract. TypeScript compiler catches any mismatches between old flat-condition usage				

and new grouped-condition format at build time.
Technology: TypeScript, Next.js type definitions.

---

13. Refactored useEntitlementAssignmentRule hook to send conditionGroups

Changed createRule and updateRule function signatures from accepting RuleConditionRequest[] to RuleConditionGroupRequest[]. The hook now sends conditionGroups in the request body instead of flat conditions. All return types, error handling, loading states, toggle, and evaluate functions remain unchanged.

Impact: All rule create and update operations from the frontend now use the grouped format ensuring interGroupOperator and negated values are properly sent to the backend.
Technology: React

hooks, TypeScript, fetch API.

---

14. Rebuilt EntitlementScriptingEditor with full group-level CRUD and scrollable layout

Replaced the flat condition list editor with a group-aware editor. Users can add, remove, and duplicate condition groups. Each group has a groupOperator dropdown (AND/OR), a negated checkbox (NOT), and an interGroupOperator dropdown rendered between groups. Conditions within each group have field, operator, and value inputs with horizontal ScrollArea to handle overflow when many conditions are added. Added fromResponse helper to convert backend conditionGroups to editor state and toRequest helper to convert editor state

back to the request format.

Impact: Users can now visually build complex multi-group rules with mixed AND/OR logic, negation, and inter-group operators directly from the UI. Previously users could only add flat conditions with a single AND/OR between each pair.
Technology: React, shadcn/ui (Select, Card, Badge, Checkbox, ScrollArea, Tooltip, Input, Button), TypeScript state management.

---

15. Rebuilt EntitlementScriptingViewer with grouped condition display and scroll support

Replaced the flat condition list viewer with a group-aware read-only display. Each group renders as a card with a left border accent showing group number, groupOperator badge, NOT badge when negated, and condition

count. Conditions within a group show field, operator, and value in inline bordered boxes with the intra-group operator badge between them. Inter-group operator badges render between group cards with divider lines. Added vertical ScrollArea with 600px max height for rules with many groups.

Impact: Administrators and auditors can now see exactly how grouped conditions are structured including which groups are negated and how groups connect to each other without needing to inspect raw JSON. Technology: React, shadcn/ui (Card, Badge, ScrollArea, ScrollBar), TypeScript.

Summary

Area	Count	Priority
Scheduler design and	2 items (scheduler	P1

		implementation	logic, global trigger)						
		Schedule type default change	1 item (IMMEDIATE to MANUAL)	P2					
		Grouped conditions initial implementation	1 item	P1					
		Testing and bug discovery	1 item (inter-group chaining bug found)	P0					
		interGroupOperator fix	3 items (DTO, SQL predicate, evidence	P0					

	builder)	
Backward compatibility	1 item (legacy condition wrapping)	P2
Build fix	1 item (listRules signature)	P0
Test data	1 item (26 employees, 7 categories)	P2
Frontend type and hook updates	2 items	P1
Frontend editor and viewer rebuild	2 items	P1

8 Apr 2026	**Entitlements Execution and Triggers**	**Frontend** 1. Removed outer Card wrapper from EntitlementHistoryViewer, tabs render directly using IconTabs with History and Zap icons 2. Extracted execution detail view into standalone `ExecutionDetailDialog.tsx` component using Dialog pattern, no extra padding 3. Standardized all badge colors to severity pattern: green (success/assigned), red (failed/error), yellow (running/revoked), gray (skipped) 4. Added `capitalize()` helper to format raw uppercase enums like EVALUATE_ALL into "Evaluate all" with underscore-to-space conversion		**8 hours**		

		5. Added info tooltips on Activity Log title, Certification History title, Assignment Runs description, execution row stats (matched/assigned/revoked/duration), and execution rows 6. Replaced two separate header buttons with single "Assignment Rules" dropdown containing "Run Now" and "Schedule" options with hover tooltips 7. Created `app/api/access-catalogue/entitlements/evaluate-all/route.ts` proxying POST to backend evaluate-all endpoint 8. Created `app/api/access-catalogue/entitlements/global-schedule/route.ts` proxying PUT to backend global-schedule endpoint 9. Created `hooks/useEvaluateAll.ts` hook managing evaluate-				

		all call with loading, error, success states and auto-clear 10. Created `hooks/useGlobalSchedule.ts` hook managing global schedule PUT call with saving, error, success states 11. Created `ScheduleConfigDialog.tsx` with 15 cron presets, custom cron input, searchable timezone picker, save/cancel, success/error feedback 12. Replaced hardcoded 19 timezone list with full IANA timezone support via shared `getTimezones()` utility, rendered as searchable Command combobox with UTC offset labels 13. Evaluate-all result banner shows below page header with green/red styling, auto-clears after 5 seconds 14. Standardized pagination on				

entitlement table and members viewer

15. Fixed client-side sorting for resourceName and assignedUsersCount columns on entitlement table
16. Removed lastUsedAt column from entitlement table
17. Removed compliance scripting tags
18. Standardized icons across history and detail views (Users, UserCheck, UserMinus, Timer, Eye, History, Zap)

Backend

19. Migrated assignment rule conditions from flat list to recursive nested condition groups with per-group AND/OR operators, inter-group operators, and group-level negation with backward compatibility
20. Fixed `triggeredBy` field: manual API calls now capture authenticated

		username via `SecurityContextHolder`, cron jobs remain "SYSTEM" 21. Added `triggeredBy` field to `RuleEvaluationEvent` so username is carried inside the event object (Spring `@TransactionalEventListener` cannot accept extra parameters) 22. Removed invalid `String username` parameter from `RuleEvaluationEventListener.handleRuleEvaluation`, reads from event instead 23. Updated `evaluateRuleById` and `evaluateAllRules` in `RuleEvaluationEngine` to accept `triggeredBy` parameter instead of hardcoding SYSTEM_TRIGGER 24. Fixed stale JPA L1 cache bug: added `entityManager.f`				

`lush()` + `entityManager.clear()` in `recalculateAllNextExecutions` before loading rules after bulk JPQL update

25. Identified cron poller `fixedRate=60000` too slow for sub-minute cron expressions, recommended reducing to 10000 for testing
26. Created `GlobalScheduleConfig` entity and `global_schedule_config` table to persist tenant-level schedule defaults (scheduleType, cronExpression, scheduleTimezone)
27. Created `GlobalScheduleConfigRepository` with `findByTenantId`
28. Created `GlobalScheduleConfigResponse` DTO for GET endpoint

29. Updated `updateGlobalScheduleType` to upsert `GlobalScheduleConfig` row alongside bulk-updating existing rules
30. Updated `createRule` to inherit schedule config from `GlobalScheduleConfig` when no scheduleType is provided in the request, falls back to IMMEDIATE if no global config exists
31. Added `getGlobalScheduleConfig` service method and GET `/global-schedule` controller endpoint
32. Changed global-schedule endpoint from `@PostMapping` to `@PutMapping`
33. Added SQL migration for `global_schedule_config` table with

		unique constraint on tenant_id				
10 Apr 2026	**Entitlements**	**Frontend Changes** 1. Resolved merge conflicts in EntitlementOverview Editor and related component files 2. Removed duplicate code blocks left over from conflict resolution 3. Added requestable yes/no select field to EntitlementOverview Editor component and passed its boolean value through onChange **Backend Changes** 1. Created EntitlementGovernanceAuditEvent record with factory methods for governance update audit events 2. Created EntitlementGovernanceAuditPublisher service to convert and publish governance audit events to Kafka 3. Created AuditEntitlementGov		**8 hours**		

ernance annotation for aspect-oriented governance audit logging

4. Created EntitlementGovernanceAuditAspect to automatically capture before/after state and publish audit events on governance updates without polluting service methods
5. Added governance audit constants to AppConstants for actions, log messages, detail keys, and metadata keys
6. Created RuleEvaluationEvent with tenantId, ruleId, triggerType, triggeredBy, and scheduleType to publish post-commit evaluation events
7. Created RuleEvaluationEventListener that triggers rule evaluation after transaction commit regardless of schedule type

		8. Added ENTITLEMENT_RULE_CREATED and ENTITLEMENT_RULE_UPDATED to TriggerType enum for explicit evaluation trigger tracking 9. Updated createRule to unconditionally publish RuleEvaluationEvent with ENTITLEMENT_RULE_CREATED trigger type after save 10. Updated updateRule to accept updatedBy parameter and unconditionally publish RuleEvaluationEvent with ENTITLEMENT_RULE_UPDATED trigger type 11. Added evaluateRuleByIdEnginePointcut targeting RuleEvaluationEngine.evaluateRuleById directly since post-commit listener bypasses the service layer 12. Added auditEvaluateRuleByI				

		dEngine around advice that intercepts engine evaluation calls for ENTITLEMENT_RULE_CREATED, ENTITLEMENT_RULE_UPDATED, and MANUAL triggers 13. Added resolveAuditUser helper that tries SecurityContext first and falls back to triggeredBy argument when context is cleared post-commit 14. Added resolveAuditTriggerSource helper to map TriggerType enum to human-readable trigger source constants 15. Added ASPECT_AUDITED_TRIGGERS set with MANUAL, ENTITLEMENT_RULE_CREATED, ENTITLEMENT_RULE_UPDATED, and EVALUATE_ALL to prevent duplicate audit events between aspect and engine				

16. Removed orphaned evaluateRulePointcut that no longer had an associated Around advice
17. Changed evaluateAllRulesPointcut to target RuleEvaluationEngine.evaluateAllRules directly to fix proxy mismatch preventing aspect interception
18. Updated evaluateAllRules audit advice to use resolveAuditUser with triggeredBy fallback for consistent actor resolution
19. Added description field to EntitlementRuleAuditEvent record as the 13th positional field between triggerSource and stateBefore
20. Added description builder methods buildRuleCrudDescription, buildRuleToggleDescription, and buildEvaluationDescription to generate

		human-readable audit summaries 21. Added resolveEvaluationBaseDescription to map trigger sources to context-aware evaluation descriptions 22. Updated all factory methods (ruleCreated, ruleUpdated, ruleDeleted, ruleToggled, globalScheduleUpdated, ruleEvaluated, entitlementAssigned, entitlementRevoked) to generate and pass description 23. Added description to buildDetails in EntitlementRuleAuditPublisher under the message key for UI rendering 24. Removed description from buildEvaluationSummary (stateAfter) to prevent it from appearing in the Event Details state view 25. Removed description from buildMetadata				

		to align with auth service audit pattern 26. Added audit description constants to AppConstants for all event types including rule CRUD, evaluation triggers, assignments, revocations, and global schedule updates				
13 Apr 2026	**Entitlements**	1. **Audit Logging Infrastructure** ◦ Created `EntitlementAuditLog` entity + DB table ◦ Enables persistence of all audit events (create, update, delete, evaluate) 2. **Repository + Query Support** ◦ Implemented `EntitlementAuditLogRepository` with filters (actor, action, date, rule) ◦ Supports pagination and flexible querying 3. **Transactional Audit Service**				

- Added `EntitlementAuditLogService` with `REQUIRES_NEW` propagation
- Ensures audit logs persist independently (even in post-commit / async flows)

4. **Audit API Exposure**
   - Introduced `GET /entitlements/{id}/audit-history`
   - Returns paginated, frontend-ready audit history
5. **DTO + Field-Level Diff**
   - Created `EntitlementAuditHistoryDto` with `changedFields`
   - Provides clean, structured diff instead of raw snapshots
6. **Engine-based Audit Persistence**
   - Moved evaluation audit logging from AOP →

`RuleEvaluationEngine`

- Ensures all execution paths (manual, cron, bulk) are covered

7. **Reliability Fixes**
   - Replaced `@Async` with `@Transactional(REQUIRES_NEW)`
   - Fixed missing logs, duplicate logs, and conditional audit skips
8. **Data Cleanup + Constants**
   - Filtered irrelevant fields from audit snapshots
   - Centralized constants (`AuditConstants`, `AppConstants`, `RuleEvaluationConstants`)
   - Improved readability and consistency

--

Frontend

Audit Diff Visualization

- Built `StateDiffDialog` with structured rendering
- Converts complex JSON into readable UI

- **Condition Rendering Logic**
  - Added `isConditionGroups()` type guard
  - Separates condition groups from simple fields
- **Manual Evaluation Access**
  - Added "Run Now" button in view mode
  - Enables evaluation without edit access
- **QueryBuilder Performance Fix**
  - Switched from controlled → uncontrolled (`defaultQuery`)
  - Prevents full re-render on each

keystroke

- **Ref-based State Handling**
  - Replaced `useState` with `useRef` for query
  - Eliminates unnecessary re-renders and focus loss
- **Controlled Remounting**
  - Introduced `builderKey`
  - Allows clean reset of QueryBuilder when needed
- **Initial Render Guard**
  - Added `isInitialChangeRef`
  - Prevents false dirty state on mount
- **UX & Stability Fixes**
  - Removed duplicate refresh calls (spinner issue)
  - Fixed duplicate props and minor

		UI inconsistencies				
`14 Apr 2026`	**Identity Detail Panel for sorting, searching for Access tab Roles and Entitlements**	**Backend** • New endpoint `GET /api/v1/identity/{identityId}/entitlements` with server-side pagination, search (displayName, description), and sorting • Resolves Identity → Shadows → Assignments → Entitlements in 4 fixed queries regardless of account count • Returns unified DTO with assignment metadata, account name, application name, entitlement catalog details • New repository methods for shadow ID lookup, batch loading, and search-filtered assignment queries via JPQL subquery • Added gateway route for `/api/v1/users/**` to auth service		**8 hours**		

(was missing, causing 404)

Frontend

- Access Tab: replaced mock data with live roles and entitlements APIs using SWR hooks
- Roles proxy route resolves identityId to userId via auth service before calling roles service
- Both sections collapsible with independent search bars (roles: client-side, entitlements: server-side with 400ms debounce)
- All columns sortable; assignment entity fields sorted server-side, joined entity fields sorted client-side
- Pagination uses standard `CustomPaginatio n` component in callback mode
- Entitlement Catalog: search and filter panel now send backend query

		params instead of client-side filtering • Filter state uses `replaceFilters` instead of merge to fix radio buttons not clearing on "All" **Problems Resolved** • Gateway returned its own 404 for `/api/v1/users/by-identity/{id}` because no route existed for `/api/v1/users/**` to auth service • Sort crashed with `UnknownPathException` when frontend sent joined entity field names (`accountName`, `entitlementName`) as sort keys on `EntitlementAssignment` entity • Roles showed "Failed to load" for identities without a linked user account; proxy route now returns empty success response instead of forwarding 404				

15 Apr 2026	**Entitlements**	1. Added proper Javadoc-style descriptions for entitlement-related methods to improve clarity in frontend integration. 2. Included missing class-level documentation under the entitlement package for better understanding of data structures. 3. Integrated business roles into the UI layer for role-based entitlement mapping. 4. Enabled display of entitlements in the Access Portal for user selection. 5. Built UI components to list available entitlements dynamically. 6. Ensured entitlements are selectable with proper state handling (checked/unchecked). 7. Reflected selected entitlements in the access scope items section.		**8 hours**		

		8. Added visibility of entitlements within scoped access views. 9. Introduced a “Details” button for each entitlement to show extended information. 10.				
`16 Apr 2026`	**Entitlements**	1. Implemented **CustomPagination** in `ObjectItemSelectorModal` (callback mode) for handling large datasets efficiently. 2. Added **page size selection** to give admins flexibility in viewing item counts. 3. Enabled **direct page navigation (go-to-page)** for faster access to specific data segments. 4. Standardized **risk badge styling** across all components using `getRiskBadgeClass`. 5. Applied `capitalize()` to ensure consistent		**8 hours**		`@Khurram Yousaf`

risk label formatting (Low/Medium/High).

6. Removed inconsistent colors and casing for risk indicators, improving **UI consistency**.
7. Introduced **risk filter** in `ObjectItemSelectorModal` for internal roles.
8. Added **riskLevel field** to role data for filtering and display purposes.
9. Enabled admins to **filter roles by risk level** during tile configuration.
10. Integrated entitlement data via gRPC to replace placeholder values like “Unknown”.
11. Displayed **real entitlement names, descriptions, and risk levels** in the UI.
12. Enhanced `AccessItemDetailsModal` to support **conditional rendering based on item type**.

13. Retained existing tab structure for roles while introducing a dedicated entitlement view.
14. Built **EntitlementViewer** with structured tabs (Overview, Attributes, Members, Risk).
15. Ensured users can **review full entitlement details before requesting access**.
16. Added **read-only mode** to `EntitlementMembersViewer` for access portal users.
17. Hid sensitive columns (Risk, Actions) and disabled row interactions in read-only mode.
18. Fixed **cart item type mapping** in `useCart.ts` for ENTITLEMENT and APPLICATION types.
19. Ensured correct **icons (e.g., Key) and type badges** display in the cart UI.
20. Added **Refresh button in**

		CartSlideout to reload latest cart data without reopening, improving user experience.				
17 Apr 2026	**Business Roles**	**Backend Identity Service** **New Package:** `access_roles_management` • Designed and implemented 3 core tables: `access_roles`, `access_role_entitlements`, `access_role_compositions` • `access_roles` stores governed roles with type, status, risk, classification, compliance tags, owner, requestable flag, approval, and certification schedule (categorical fields as VARCHAR for tenant flexibility) • `classification` and `compliance_tags` stored as JSONB arrays with GIN indexes for multi-label querying		**8 hours**		@Khurram Yousaf

- `access_role_entitlements` implements many-to-many mapping between roles and entitlements with duplicate prevention
- `access_role_compositions` supports parent-child role hierarchy with BFS-based circular dependency detection
- Full CRUD REST APIs implemented for roles with:
  - Create role
  - Get by ID (with entitlements + parent/child roles)
  - List with filters (type, status) + pagination
  - Update role
  - Delete role
- Separate service layers for roles, entitlements, and compositions to avoid monolithic services
- MapStruct used for clean entity ↔ DTO mapping with null-safe partial updates

- Unified structured logging across services (category, message, key-value detail pairs)
- Standardized `ApiResponse<T>` wrapper across all endpoints
- DB migration:
  - Converted `classification` from VARCHAR → JSONB array
  - Added `compliance_tags` as JSONB column
- Research conducted on hierarchy cycle detection approaches:
  - Evaluated BFS, recursive CTE, closure table, nested sets, Dgraph
  - Final approach: BFS (simpler), with recommendation to move to recursive CTE for scale

---

Frontend Next.js

- Created `types/access-`

`roles.ts` aligning frontend types with backend DTOs and `ApiResponse` structure

- Built API proxy routes under `/api/access-catalogue/roles`:
  - POST/GET for roles list
  - GET/PUT/DELETE for role by ID
- Implemented `use-create-access-role` hook with loading and error state handling
- Implemented `use-access-roles` hook for paginated role fetching
- Created `role-adapter.ts` to map backend `AccessRoleResponse` into UI-friendly Role model
- Refactored Add Role Wizard into modular tabs:
  - `BasicInfoTab`
  - `OwnershipGovernanceTab`
  - `EntitlementsTab`

- `RiskComplianceTab`
- `ReviewSubmitTab`

- Integrated real identity search API for owner selection with pagination support
- Replaced mock entitlement data with real entitlement API (search + pagination support)
- Introduced `BadgeEditor` for classification and compliance tag management (add/remove UI)
- Added `CharacterCounter` component for enforcing 256-character limits on name/description fields
- Updated Roles page to:
  - Fetch real API data
  - Use adapter for transformation
  - Show loading, error, and empty states

		◦ Remove all mock data • Added auto-refresh after successful role creation --- **Business Impact** • Roles catalog is now fully persisted and production-ready in the IGA platform • Admins can define governed roles with ownership, entitlements, risk, and compliance metadata • Role hierarchy is supported with safeguards against circular dependencies • Enables foundation for: ◦ Role-based access requests ◦ Certification workflows ◦ Dynamic entitlement assignment ◦ Governance-driven access control model				

20 Apr 2026	**Business Roles**	**Backend** 1. Added `businessFunction` field to the access role entity, request/response DTOs, and database schema 2. Added `POST /{roleId}/entitlements` endpoint to link entitlements to a role 3. Added `DELETE /{roleId}/entitlements` endpoint to unlink entitlements from a role 4. Added `GET /{roleId}/entitlements/grouped` endpoint to fetch role entitlements grouped by application with two-level pagination (app-level + entitlement-level) 5. Added `ManageRoleEntitlementsRequest` DTO for add/remove entitlement operations 6. Added `GroupedRoleEntitlementsDto`				

response structure with nested pagination metadata

Frontend

1. Created Next.js API route `roles/[id]/entitlements/route.ts` splits `{ addIds, removeIds }` into separate `POST` and `DELETE` calls to backend
2. Created `useSyncRoleEntitlements` hook for syncing entitlement links with loading/error state
3. Created `useGroupedRoleEntitlements` hook to fetch entitlements grouped by application with pagination
4. Built `RoleEntitlementsEditor` component displays entitlements grouped by application, supports add/remove via `EntitlementCata`

`logPicker`, tracks original IDs for diff computation on save

5. Updated `RoleEditor.handleSave` to compute entitlement diff (added/removed) and sync after metadata PUT
6. Built owner selector in `RoleEditor` fetches identities via `useIdentities` hook, displays owner name/email/status, supports search and identity type filtering with pagination
7. Fixed `role-adapter.ts` `classification` preserved as `string[]`, `riskLevel` mapped to both `riskLevel` and `risk.score`, `complianceTags` passed through, `ownerId` set separately
8. Fixed `RoleOverviewEditor` classification tags rendered and edited as

`string[]` via `BadgeEditor`, added `CharacterCounter` and `maxLength` on inputs, added `businessFunction` field

9. Fixed `RoleRiskComplianceEditor` compliance tags (separate from classifications) editable via `BadgeEditor`, risk level sent as-is without lowercasing
10. Fixed `buildPayload` sends `classification` as `string[]`, `complianceTags` as `string[]`, `riskLevel` without case transformation
11. Fixed classification badge in `RoleViewer` renders `string[]` as multiple badges
12. Added post-save refresh flow `RoleDetailView` re-fetches role detail from backend after

		save, converts via adapter, pushes fresh data to Page which updates `selectedRole` and refreshes sidebar list 13. Extracted `RoleEditorHeader` into its own component for cleaner code structure				
`21 Apr 2026`	**Business Roles Child Roles, Graphs, Heirarchy**	Backend • **Partial success for child role composition** - `addChildRoles` no longer throws on individual failures (circular dependency, self-reference); processes each child independently and returns added/skipped/failed lists • **New `AddChildRolesResultDto`** - response DTO with per-child success/failure details enabling granular error reporting to the UI • **Controller returns HTTP 207 on partial**				

failure - frontend can distinguish full success (200) from partial success (207) and display per-child error messages

- **New `GET /{roleId}/parents` endpoint** - exposes parent role lookup for hierarchy visualization in the graph
- **Entitlement count enrichment on child/parent role responses** - `getChildRoles` and `getParentRoles` now return `entitlementCount` per role, eliminating N+1 queries on the frontend
- **New `RESPONSE_CHILDREN_PARTIAL` and `RESPONSE_PARENTS_FETCHED` constants** - consistent API response messaging
- `AccessRoleCompositionServiceImpl` **now injects**

`AccessRoleEntitlementRepository` - enables batch entitlement count lookups in composition queries

Frontend

- **Fixed graph node overlap** - increased spacing constants, switched entitlements to a grid layout (3 columns), child row Y is now dynamically calculated below the entitlement grid
- **Fixed graph zoom** - added `fitViewOptions` with `padding: 0.3` and `maxZoom: 1`, preventing over-zoom on small graphs; added `minZoom`, `maxZoom`, and `defaultViewport` for consistent UX
- **Graph re-fits on role/view mode change** - added `key` prop to `ReactFlow` forcing remount and re-fit when switching roles or view modes

- **Parent roles displayed in relationship graph** - new `useParentRoles` hook, Next.js route (`/api/.../parents`), and purple "parent of" edges above the selected role node
- **Fixed child roles not displaying in RoleRelationshipBuilder** - race condition fix: `useChildRoles` now starts with `loading: true` , preventing premature empty-state render
- **Real child role CRUD in editor** - `RoleRelationshipBuilder` rewritten to use `useChildRoles` for fetching and `useSyncRoleChildren` for saving, with paginated role picker
- **Partial success handling for child role sync** - failed children (circular dependency) show per-child toast errors

with role name and reason; successfully added children are kept; failed ones are reverted from UI

- **New `useSyncRoleChildren` hook** - returns `added`, `skipped`, `failed`, `removedCount`, `removeError` for granular error handling
- **New Next.js children sync route** - `POST /api/.../children` fans out to backend POST/DELETE, parses partial success (207) responses
- **New `useParentRoles` hook and `/api/.../parents` route** - fetches parent roles from backend for graph and editor display
- **Admin Role classification fix in `UMLRoleNode`** - now handles `string[]` classification (was

breaking on arrays with `=== 'Admin Role'`)

- **Status field added to Add Role wizard** - dropdown with Enabled/Disabled in BasicInfoTab; status included in Review & Submit summary
- **Entitlement selection now required** - Next button on Entitlements step disabled until at least one entitlement is selected
- **Child roles shown in Review & Submit tab** - displays count of selected child roles before submission
- **Role status normalized to `enabled` / `disabled`** - updated `Role` type, adapter (`role-adapter.ts`), hierarchy tree grouping, and all status display points
- **Status display capitalized everywhere** - UML nodes, hierarchy tree, and owner badges

		now show "Enabled"/"Disabled" with proper casing				
22 Apr 2026	**Access Roles Auditing**	Frontend • **Role Management UI Enhancements** ◦ Updated role management flows to support **child role retrieval and visualization** in role composition views. ◦ Modified role-related routes and client hooks to align with updated role management APIs (including nested child-role handling and entitlement synchronization). ◦ Improved payload handling for role create/update operations to support hierarchical role relationships and audit-compatible request structures. • **State Management / API Integration** ◦ Integrated updated hooks for role		**8 hours**		

CRUD and sync endpoints to support:
    - Role metadata updates
    - Entitlement assignment synchronization
    - Child role composition updates
  - Refined frontend handling for desired-state sync requests, reducing client-side inconsistency in entitlement/child role assignment workflows.
- **UX / Logic Improvements**
  - Improved nested role relationship rendering for roles containing child roles with their own descendants.
  - Standardized client-side interaction around “single user action = single update event,” aligning UI actions with

backend audit event boundaries.
    - Reduced risk of duplicate or fragmented update requests during multi-entity role updates.
- **Bugs Fixed**
    - Fixed child role retrieval issues where nested role relationships were not consistently surfaced due to incomplete API response handling.
    - Fixed entitlement/child role update inconsistencies caused by mismatched frontend desired-state payloads vs backend diff-processing expectations.
    - Resolved audit-trigger alignment issue where frontend actions could generate fragmented update behavior instead of a consolidated audited operation.

Backend

- **APIs Developed / Modified**
  - Enhanced access role management endpoints for:
    - Child role retrieval
    - Role composition updates
    - Entitlement synchronization
    - Audit-aware role create/update/delete operations
  - Updated role management routes to integrate global auditing across role lifecycle operations.
- **Database Changes**
  - Created and audited tables supporting business roles and role audit event persistence.
  - Refined schema support for hierarchical role relationships (roles with nested child roles and

entitlement mappings).

- Structured audit persistence to capture:
  - `ROLE_CREATED`
  - `ROLE_UPDATED`
  - `ROLE_DELETED`
  - `ENTITLEMENTS_UPDATED`
  - `CHILDREN_UPDATED`

- **Business Logic Implemented**
  - Implemented consolidated audit model enforcing **one user action = one audit event**.
  - Added backend diff computation for entitlement and child-role updates, capturing adds/removes within a single event instead of fragmented action records.
  - Enhanced audit event structure to include richer `stateAfter` / change payloads including names, risk levels,

relationship changes, and failed child-role operations.

- **Integrations**
  - Integrated auditing globally using **Spring AOP (Around Advice / Pointcuts / Join Points)** across service-layer operations.
  - Wired auditing into controller → service → implementation flow as a cross-cutting concern without polluting domain logic.
- **Performance / Reliability Optimizations**
  - Reduced audit event noise by consolidating multiple sub-operations into single transactional audit records.
  - Improved consistency of audit capture by moving event generation closer to service-layer

transactional boundaries.
    - Reduced risk of partial audit records during nested role updates involving children and entitlements.
- **Bugs Fixed**
    - Fixed audit gaps where nested child roles and inherited relationships were not fully captured in role events.
    - Fixed incorrect handling of entitlement and child-role changes where add/remove operations could be split or incompletely audited.
    - Resolved audit structure issues in role updates causing incomplete event payloads for downstream audit consumers.

---

2. Business Impact (Metrics & Reasoning)

- **Problem Solved**

- Addressed incomplete auditability for hierarchical access role operations, particularly around nested child roles and entitlement changes.
- Solved fragmentation where one business action could produce multiple audit artifacts or inconsistent records.

- **Why This Was Important**
  - Improved compliance-grade traceability for access governance changes.
  - Reduced operational risk in RBAC change tracking and audit investigations.
  - Enabled accurate tracking of complex role composition updates for governance and risk review.

- **Measurable Outcomes / Enablement**
  - Reduced audit event fragmentation from multiple sub-events to **1 consolidated event per user action**.
  - Increased audit coverage for nested role relationship changes to include previously missing child-role lineage data.
  - Enabled new capability: full auditing for hierarchical role composition changes and entitlement synchronization.
  - Reduced likelihood of incomplete audit records during multi-entity updates.
- **Trade-offs / Decisions**
  - Chose consolidated audit actions (5 actions

vs granular 7+) to reduce event noise and preserve business-level semantics.
    - Kept diff computation in backend rather than client to ensure authoritative change detection and audit consistency.
    - Implemented auditing via AOP rather than embedding logic inside domain services to keep cross-cutting concerns isolated.

---

3. Risks / Pending Work

- **Known Issues / Limitations**
    - Audit payload size may grow for deeply nested role hierarchies.
    - Complex child-role recursion paths may need additional validation to

prevent edge-case inconsistencies.
    - Failed child-role operation handling may need expanded downstream visibility/monitoring.
- **Dependencies / Blockers**
    - Dependent on validation from audit consumers/compliance stakeholders on event payload structure.
    - Requires integration-level verification for downstream systems consuming audit events.
- **Next Steps**
    - Add test coverage for deep nested role hierarchies and recursive relationship scenarios.
    - Validate audit event payloads against reporting/compliance requirements.

		◦ Review indexing/query strategy for audit tables as event volume scales. ◦ Complete monitoring/alerting for failed child-role update scenarios.				
23 Apr 2026	**Roles Catalogue**	1. Integrated audit logs into the frontend to display history of audit events for child role updates, entitlement changes, and attribute modifications. 2. Enhanced audit history visibility by surfacing role-related change events in a structured and user-accessible format. 3. Improved parent role relationship visualization in the graph to provide clearer hierarchy representation and relationship tracing. 4. Refined graph behavior and data handling for parent-child role relationships to		**8 hours**		

		improve consistency and usability. 5. Standardized pagination behavior across shared components to align with common pagination patterns and improve consistency. 6. Fixed pagination-related issues in affected components, including navigation behavior and data rendering alignment. 7. Rebased the feature branch with the latest development branch updates to keep changes aligned with the current codebase. 8. Resolved merge conflicts during rebase and validated integration across impacted modules. 9. Deployed the updated implementation for validation and verified the integrated changes in the development environment.				

		10. Completed end-to-end testing in development covering audit history, role graph enhancements, pagination updates, and post-rebase stability checks.				
24 Apr 2026	**Roles Catalogues Fetch**	**Backend** 1. Introduced a lightweight directory summary endpoint to support role hierarchy navigation using aggregated data instead of full role payloads. 2. Added `PATH_DIRECTORY_SUMMARY` along with supporting response and logging constants in `AccessRoleConstants`. 3. Added `getRoleDirectorySummary(String tenantId)` to `AccessRoleService` to provide summarized hierarchy data. 4. Implemented directory summary		**8 hours**		

aggregation logic in `AccessRoleServiceImpl`.

5. Added `RoleDirectoryGroupSummaryProjection` to support grouped hierarchy data retrieval at the repository layer.
6. Added `findRoleDirectoryGroupSummary(String tenantId)` to group roles by `businessUnit` and `type`.
7. Added `AccessRoleDirectoryGroupSummaryDto` to represent grouped counts at the role type level.
8. Added `AccessRoleDirectoryBusinessUnitSummaryDto` to represent grouped hierarchy data by business unit.
9. Added `AccessRoleDirectorySummaryResponseDto` as the top-level response

object for directory summary data.

10. Exposed a new endpoint in `AccessRoleController` to return grouped business unit and role type counts.
11. Standardized null or blank business units as `Unspecified` in the service response to ensure consistent grouping.
12. Optimized hierarchy initialization by returning summary counts and grouping data only, excluding full role records from the initial response.

Frontend

1. Identified that the current hierarchy implementation loads excessive role data into memory to build the tree.
2. Identified that grouping by `businessUnit` and `type` is currently performed entirely on the frontend, creating

unnecessary processing overhead.

3. Identified that search and filtering are currently in-memory operations and should move to a backend-driven model.
4. Confirmed that `RoleHierarchyTree` currently renders grouped hierarchy using the full fetched role dataset.
5. Defined a new approach where hierarchy initialization begins with a backend-driven grouped summary response.
6. Defined that the frontend should use the new directory summary API to retrieve business unit and type bucket data as the first hierarchy request.
7. Defined that the hierarchy view should no longer depend on full role payloads for top-level rendering.

		8. Defined a next-phase enhancement to support paginated or lazy loading of roles within each business unit and type group on expansion. 9. Defined that full role details should remain a separate fetch path and should not be included in hierarchy bootstrap requests. 10. Defined the optimization path as summary-first initialization followed by per-group pagination to support scalability.				
27 Apr 2026	**Access Roles**	**Frontend** • Added server-side role search in navigator for accurate, scalable role discovery. • Added server-side pagination for group roles using the shared pagination component. • Added Apply Filters flow so status and risk filters only execute when user confirms.				

		• Moved filtering responsibility from client-side processing to backend-driven requests. • Simplified filter panel to only status and risk level to avoid conflicting controls. • Kept business unit and role type selection in tree navigation for clearer user experience. • Added refresh action in hierarchy to reload the latest summary and role data. • Fixed infinite re-fetch loop by stabilizing hook dependencies and request parameters. • Preserved toolbar actions during loading and limited loading and empty states to the roles section. • Wired intelligence analytics UI to real APIs instead of mock dashboard aggregates.				

- Connected analytics drilldown modals to real paginated role data using the existing roles hook.
- Reduced search field width and constrained it with `max-w-xs` to improve layout balance.
- Replaced rigid grid with flexible row layout to remove excessive spacing between filters.
- Grouped status, department, and export actions into a single aligned control cluster for consistent spacing.
- Fixed the Identities page header for improved alignment and consistency.
- Added Created At and Synced At columns to the Identities page.
- Added custom pagination implementation for the Identities page to align with expected behavior and UX standards.

		Backend • Added paginated roles listing with server-side search support. • Added multi-value server-side filters for role type, status, business unit, and risk level. • Updated summary endpoint to support server-side filtering by status, risk level, and search. • Enabled hierarchy tree to reflect filtered backend summary instead of static grouping. • Standardized frontend and backend query contract to plural list-based parameters. • Added and integrated analytics APIs for KPI cards, role-type distribution, and business-unit charts. • Exposed data needed for frontend drilldowns to fetch real role records through the existing listing API.				